

GraphHunter

Referencia de arquitectura al estilo CLRS

Lucas Sotomayor
Base4Security

17 de abril de 2026

Índice general

Prefacio	VI
I Preliminares	1
1. Introducción	2
1.1. Dominio del problema	2
1.2. Vista general del sistema	2
1.3. Qué cubre esta referencia	3
2. Notación y preliminares	4
2.1. Grafos	4
2.2. Caminos e hipótesis	4
2.3. Notación asintótica	5
2.4. Modelo de memoria	5
2.5. Convención de punteros a fuente	5
II Estructuras de datos principales	6
3. La codificación de CompactRelation	7
3.1. Layout	7
3.2. Codificación MVCC	8
3.3. ¿Por qué no <code>#[repr(C, packed)]?</code>	8
4. El internador de strings	9
4.1. Estructura	9
4.2. Algoritmos	9
4.3. Métricas	10
4.4. El punto de costura (<i>seam</i>) de trait	10
5. El grafo en memoria	12
5.1. Campos	12
5.2. Algoritmos de inserción	12
5.3. Finalización	13
6. El Spillable Edge Store	15
6.1. Modos	15
6.2. Append	15
6.3. Finalización vía merge <i>k</i> -way	16
6.4. Lecturas	17

7. El Metadata Store	18
7.1. Estructura	18
7.2. Operaciones	18
7.3. Inmutabilidad	19
8. El AST de hipótesis	20
8.1. Tipos	20
8.2. Invariantes	21
8.3. Chequeo estático de compatibilidad	21
III Algoritmos principales	22
9. Parseo del DSL	23
9.1. Gramática	23
9.2. Algoritmo	23
10. Matcheo de patrones temporales	25
10.1. Estado compartido	25
10.2. Entry point: SEARCH-TEMPORAL-PATTERN	26
10.3. El DFS iterativo	26
10.4. La variante smart	28
10.5. Paralelismo	28
11. Aceleración SIMD	30
11.1. Puente FFI de libgraphmatch	30
11.2. Intersección AVX2 puro-Rust	31
11.3. Roadmap	31
12. Scoring de anomalía	34
12.1. Estado y finalización	34
12.2. Concentración de vecindad	35
12.3. Puntuar un camino	35
12.4. El <i>seam</i> de trait (P2-A)	36
12.5. Estimación rápida a nivel nodo	36
13. Centralidad y analíticos	38
13.1. Centralidad betweenness (Brandes con sampling)	38
13.2. PageRank temporal	40
13.3. Schema de relaciones	40
14. Deduplicación y paginación de caminos	42
14.1. Modos de dedup	42
14.2. Algoritmo	42
14.3. Estabilidad del sort	44
14.4. Contrato de paginación	44
15. Diff de hunts	45
15.1. Variante clásica por tiempo de evento	45
15.2. Trampa semántica: datos que llegan tarde	45
15.3. La variante por snapshot (diferida)	46
15.4. Validación de entrada	46

IV Pipeline de ingesta	47
16.El trait LogSource	48
16.1. Estructura	48
16.2. Implementaciones	48
17.Parsers	50
17.1. Trait	50
17.2. Implementaciones	50
17.3. Despacho de parser	50
18.Writer y backpressure	52
18.1. Forma del pipeline	52
18.2. Módulos	52
18.3. Contención de locks	52
19.Estampado MVCC a nivel arista	54
19.1. Estado actual	54
19.2. Trabajo pendiente	54
20.Persistencia de desborde	55
20.1. Invariantes	55
20.2. Formato de archivo	55
V Puntos de extensión	56
21.Traits GraphRead / GraphWrite	57
21.1. Traits	57
21.2. Qué va en el trait y qué no	57
21.3. Contrato dual-impl	58
22.Traits Scorer y ScoreComponent	59
22.1. Traits	59
22.2. Orquestador compuesto	59
22.3. Migración	60
23.Trait StringInternerBackend	61
23.1. Contrato	61
23.2. Condiciones disparadoras	61
24.Trait IngestAdapter	62
24.1. Contrato	62
24.2. Implementación genérica	62
24.3. Propagación MVCC	62
25.Cargador de catálogo YAML	64
25.1. Precedencia	64
25.2. Validación	64
25.3. Diagnósticos	64

VI	Capa de transporte	65
26.	La fachada canónica de API	66
26.1.	Estado	66
26.2.	Módulo de operaciones	66
26.3.	Handle de sesión	66
27.	DTOs	67
27.1.	Layout del módulo	67
27.2.	Patrones comunes	67
27.3.	Invariantes de forma	67
28.	Adaptadores de transporte	68
28.1.	Tauri	68
28.2.	HTTP	68
28.3.	MCP	69
28.4.	EventEmitter	69
29.	Arnés de paridad	70
29.1.	Fase A — forma de DTO	70
29.2.	Fase B — escenario	70
29.3.	Fase C diferida	71
30.	Persistencia de sesión	72
30.1.	Layout en disco	72
30.2.	Forma del SessionFile	72
30.3.	Progreso de carga	72
30.4.	Migración v2 (diferida)	73
A.	Resumen de complejidad	74
B.	Referencia del catálogo	75
C.	Benchmarks	76
C.1.	Ejecución	76
C.2.	Baseline actual (ilustrativo)	76
C.3.	Comparación three-way SIMD (planeada)	76
D.	Índice de fuente	77
D.1.	graph_hunter_core/src/	77
D.2.	graph_hunter_api/src/	78
D.3.	app/src-tauri/	78
D.4.	graph-hunter-mcp/	79
D.5.	libgraphmatch/	79

Prefacio

Este documento describe la arquitectura y los algoritmos de GraphHunter, un motor de *threat hunting* sobre grafos temporales. El estilo está modelado sobre *Introduction to Algorithms* (Cormen, Leiserson, Rivest, Stein — “CLRS”): cada algoritmo se presenta como pseudocódigo formal con líneas numeradas, seguido de la derivación de su complejidad temporal y espacial y, donde los invariantes no son obvios, un argumento de corrección.

La referencia es interna: su propósito es que un futuro mantenedor (o el autor, dentro de seis meses) pueda entender *por qué* el motor tiene la forma que tiene sin leer cada archivo fuente. Cada capítulo termina con un bloque *En el código* que da punteros `archivo:línea` hacia la implementación Rust, para bajar a la fuente en cualquier punto de interés.

Las Partes I–III están desarrolladas en esta revisión. Las Partes IV–VI (pipeline de ingesta, puntos de extensión, capa de transporte) se entregan como *stubs* con estructuras y referencias de archivo pero sin pseudocódigo completo; la intención es expandirlas capítulo por capítulo en revisiones posteriores.

Parte I

Preliminares

Capítulo 1

Introducción

1.1. Dominio del problema

GraphHunter es una herramienta interactiva de *threat hunting* construida alrededor de una idea: la mayor parte del comportamiento de un atacante es naturalmente un camino en un grafo dirigido temporal, así que dejamos al analista *pedir caminos* en vez de escribir SQL por tabla. Una hipótesis se expresa en un pequeño DSL de cadena de flechas — por ejemplo,

```
User -[Auth \{status="Failure"\}]-> IP HAVING count(distinct IP) >= 3 WITHIN 24h
```

y el motor devuelve los caminos del grafo ingerido que la satisfacen, rankeados por un puntaje de anomalía compuesto. El grafo se construye a partir de fuentes de logs heterogéneas (Microsoft Sentinel, Defender XDR, Fortinet, Sysmon, IIS, CSV/JSON genéricos). Las cargas típicas van desde unos pocos miles de aristas (un ticket de incidente) hasta cientos de millones (dump EVTX de varias semanas de una empresa mediana).

1.2. Vista general del sistema

El código está organizado como un monorepo Rust con cinco componentes principales, dispuestos en capas como en la Figura 1.1.

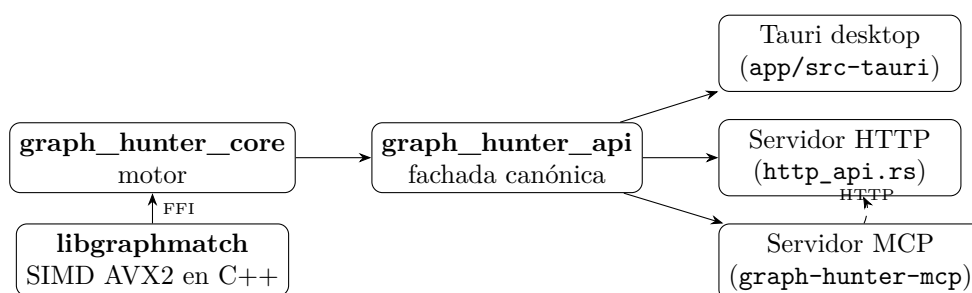


Figura 1.1: Grafo de crates. El motor (**graph_hunter_core**) lo consume una fachada canónica (**graph_hunter_api**); tres capas de transporte delegan en esa fachada. El servidor MCP llega al mismo motor invocando el servidor HTTP sobre la interfaz de loopback, en vez de enlazarse directamente a la fachada.

El motor es una biblioteca Rust pura: sin dependencias de framework, sin estado global salvo un puñado de catálogos cacheados vía `OnceLock`. La crate fachada (**graph_hunter_api**) envuelve al motor con gestión de sesión, DTOs y un trait `EventEmitter`, de modo que las tres capas de transporte compartan una única implementación sin importar los tipos de bajo nivel. El backend SIMD en C++ (**libgraphmatch**) se enlaza por FFI cuando el feature `simd` está activado; una alternativa pura en Rust (**simd_rust**) ya está en camino de reemplazarlo (Capítulo 11).

1.3. Qué cubre esta referencia

El documento recorre el motor de abajo hacia arriba:

- **Parte II** — las estructuras de datos que el motor mantiene en memoria: relaciones compactas, el internador de strings bidireccional, el grafo en memoria, el edge store con capacidad de *spill*, el almacén externo de metadatos y el AST de hipótesis que produce el parser del DSL.
- **Parte III** — los algoritmos principales: parseo del DSL, matcheo de patrones temporales (DFS con podado, variante *smart* con cota de anomalía), aceleración SIMD, *scoring* de anomalía, analíticos de centralidad (betweenness de Brandes, PageRank temporal), deduplicación & paginación de caminos, y *diff* de hunts.
- **Parte IV** — el pipeline de ingesta en *streaming*: trait `LogSource`, parsers, escritor con canales acotados, estampado MVCC a nivel de arista y persistencia de desborde.
- **Parte V** — los puntos de extensión que introdujo el refactor P2: `GraphRead/GraphWrite`, `ScoreComponent/Scorer`, `StringInternerBackend`, `IngestAdapter` y el cargador de catálogo YAML.
- **Parte VI** — la capa de transporte: la API canónica, los DTOs, `EventEmitter`, los adaptadores Tauri/HTTP/MCP, el arnés de paridad y la persistencia de sesión.

Cierran el documento cuatro apéndices: un resumen de complejidad de una página (Ap. A), la referencia del catálogo ATT&CK (Ap. B), una tabla de benchmarks (Ap. C) y un índice de código que mapea cada algoritmo a su `archivo:linea` (Ap. D).

En el código.

Las raíces de crates son `graph\_hunter\_core/Cargo.toml`, `graph\_hunter\_api/Cargo.toml`, `app/src-tauri/Cargo.toml`, `graph-hunter-mcp/package.json` y `libgraphmatch/CMakeLists.txt`.

Capítulo 2

Notación y preliminares

Este capítulo fija la notación usada en todo el documento. Las convenciones son compatibles con CLRS donde aplica.

2.1. Grafos

Definición 2.1 (Multigrafo dirigido temporal). Un *multigrafo dirigido temporal* es una tupla $G = (V, E, \tau, \ell_V, \ell_E)$ donde V es un conjunto finito de vértices, $E \subseteq V \times V \times \mathbb{N}$ es un multiconjunto de aristas dirigidas (el tercer componente es un identificador único de arista, por lo que varias aristas entre el mismo par son elementos distintos de E), $\tau : E \rightarrow \mathbb{Z}$ asigna a cada arista una marca temporal de evento, $\ell_V : V \rightarrow \mathcal{T}_V$ etiqueta a los vértices con un tipo de entidad, y $\ell_E : E \rightarrow \mathcal{T}_E$ etiqueta a las aristas con un tipo de relación.

Escribiremos $|V| = n$ y $|E| = m$. La vecindad saliente de un vértice $v \in V$ en la ventana $[t_0, t_1]$ es

$$N_{[t_0, t_1]}^+(v) = \{u \in V \mid \exists e = (v, u, k) \in E : \tau(e) \in [t_0, t_1]\}.$$

La vecindad entrante $N_{[t_0, t_1]}^-(v)$ se define simétricamente. Cuando la ventana es todo el dominio omitimos el subíndice y escribimos $N^+(v)$, $N^-(v)$.

Definición 2.2 (Extensión MVCC). La *extensión MVCC* de G agrega un segundo timestamp $\iota : E \rightarrow \mathbb{Z}$ que registra cuándo se insertó cada arista en el grafo. El motor almacena ι como un delta compacto contra un epoch fijo; ver Capítulo 3. Una consulta por snapshot en el instante s considera sólo el subgrafo

$$G|_s = (V, \{e \in E \mid \iota(e) \leq s\}, \tau, \ell_V, \ell_E),$$

es decir, “lo que el grafo sabía al momento s ”.

2.2. Caminos e hipótesis

Definición 2.3 (Caminos temporales). Un *camino temporal* de longitud ℓ en G es una secuencia

$$\pi = (v_0, e_1, v_1, e_2, \dots, e_\ell, v_\ell),$$

con $v_i \in V$, $e_i = (v_{i-1}, v_i, k_i) \in E$, y $\tau(e_1) \leq \tau(e_2) \leq \dots \leq \tau(e_\ell)$ (monotonía temporal). Con frecuencia abreviaremos un camino como su lista de vértices $(v_0, v_1, \dots, v_\ell)$ cuando las aristas intermedias se deducen del contexto.

Definición 2.4 (Caminos k -simple). Un camino es *k -simple* si y sólo si ningún vértice aparece en él más de k veces. La k -simplicidad relaja la exigencia clásica de no-repeticiones ($k = 1$) para admitir ciclos que un atacante real sí exhibe — por ejemplo, un bucle de callback C2 donde el mismo proceso vuelve a entrar en el camino. k se fija por hipótesis; el valor por defecto es 1.

Definición 2.5 (Hipótesis). Una *hipótesis* H es una cadena de flechas

$$H = (\sigma_0, \rho_1, P_1^{\text{edge}}, P_1^{\text{dst}}, \sigma_1, \rho_2, \dots, \rho_\ell, P_\ell^{\text{edge}}, P_\ell^{\text{dst}}, \sigma_\ell)$$

donde $\sigma_i \in \mathcal{T}_V \cup \{*\}$ es el tipo de entidad requerido en el i -ésimo vértice (o $*$ para “cualquiera”), $\rho_i \in \mathcal{T}_E \cup \{*\}$ es el tipo de relación requerido en la i -ésima arista, y $P_i^{\text{edge}}, P_i^{\text{dst}}$ son predicados opcionales sobre los metadatos de la arista y del vértice de destino, respectivamente. La cadena también lleva restricciones de agregación (**HAVING**, **WITHIN**) que se procesan después del match. Un camino $\pi = (v_0, \dots, v_\ell)$ *satisface* H si y sólo si $\ell_V(v_i) \in \sigma_i$, $\ell_E(e_i) \in \rho_i$, y cada predicado se cumple sobre la arista o el vértice correspondiente.

2.3. Notación asintótica

Valen las convenciones estándar de CLRS. $O(f(n))$ es cota superior, $\Theta(f(n))$ es cota ajustada, $\Omega(f(n))$ es cota inferior. “Amortizado $O(f)$ ” significa que el costo por operación, promediado sobre una secuencia peor caso, es $O(f)$, aun cuando operaciones individuales sean más caras.

2.4. Modelo de memoria

El modelo de propiedad (*ownership*) de Rust hace que la mayoría de los invariantes sean chequeables localmente. Donde el motor depende de mutabilidad interior, lo marcamos explícitamente — `Arc<RwLock<_>` para estado de sesión compartido, `RwLock<SpillableEdgeStore>` para el buffer de aristas en memoria que puede volcarse a disco durante ingesta, `OnceLock` para el snapshot del catálogo a nivel proceso. El motor nunca comparte grafos mutables entre threads sin un lock; el matcher DFS paraleliza dando a cada worker rayon una referencia inmutable y una pila scratch local al thread (Capítulo 10).

2.5. Convención de punteros a fuente

Cada capítulo termina con un bloque *En el código* que lista punteros `archivo:línea` a la implementación Rust. Las rutas son relativas a la raíz del repositorio, p. ej. `graph\_hunter\_core/src/graph.rs:461`. Los números de línea pueden derivar entre revisiones, así que si el puntero está desactualizado conviene grepear el nombre de la función.

Parte II

Estructuras de datos principales

Capítulo 3

La codificación de CompactRelation

La estructura de datos más densa del motor es la representación en memoria de las aristas. Un grafo de 200 M aristas a ~200 bytes cada una (el **Relation** completo con metadatos, strings, etc.) ocuparía 40 GB — por eso el motor guarda cada arista dos veces: una como **Relation** rica, usada sólo mientras la arista se está insertando, y de ahí en más como **CompactRelation** de 32 bytes, que vive en la lista de adyacencia el resto de la sesión.

3.1. Layout

CompactRelation es `\#[repr(C)]`, con los campos ordenados por mayor a menor alineación para evitar padding interior. La Figura 3.1 muestra el layout byte a byte.

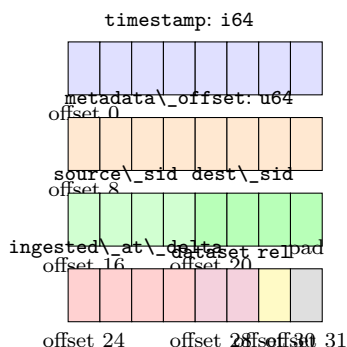


Figura 3.1: Layout de memoria de **CompactRelation** (32 bytes). El u32 `ingested_at_delta` (P3) recupera 4 de los 5 bytes de padding final del layout pre-P3.

Semánticamente:

- **timestamp**: tiempo de evento de la arista, en segundos epoch.
- **metadata_offset**: offset dentro del **MetadataStore** externo (Capítulo 7); cero significa “sin metadatos”.
- **source_sid**, **dest_sid**: manejadores de 32 bits al **StringInterner** (Capítulo 4).
- **ingested_at_delta**: segundos desde el epoch MVCC base (ver abajo); cero en aristas heredadas.
- **dataset_tag**: manejador de 16 bits a la tabla **Arc<str>** de identificadores de dataset por grafo; cero significa “sin dataset”.
- **rel_type_tag**: enum tag de 8 bits para el tipo de relación.

El `rel\_type\_tag` es deliberadamente `u8` para que el matcher pueda hacer comparaciones SIMD sin ramas (Capítulo 11).

3.2. Codificación MVCC

Invariante (Tamaño MVCC). `size\_of::<CompactRelation>() == 32`. Esto se asevera en `tests/relation\_`. Violarlo cuesta $\sim 20\%$ del footprint del grafo en memoria, así que es una aserción cargada de peso.

El delta MVCC se mide contra `INGEST\_EPOCH\_BASE = 1 704 067 200` segundos (2024-01-01 UTC). Un `u32` cubre $\approx 2^{32}$ segundos, unos 136 años; el motor satura en `u32::MAX` para ingestas más allá de ~ 2160 . Las aristas heredadas cargadas desde un archivo de sesión pre-P3 resuelven al propio epoch base, que es un instante definido aunque impreciso.

El Algoritmo 3.1 muestra el codificador. La decodificación es el trivial `INGEST\_EPOCH\_BASE + delta` as `i64`.

Algorithm 3.1: CODIFICAR-INGESTED-AT

Entrada: timestamp absoluto en segundos epoch, t

Salida: delta `u32` a guardar en `CompactRelation`

```

1 si  $t \leq \text{INGEST\_EPOCH\_BASE}$  entonces
2   | devolver 0;
3 fin
4  $\delta \leftarrow t - \text{INGEST\_EPOCH\_BASE}$ ;
5 si  $\delta > \text{u32::MAX}$  entonces
6   | devolver u32::MAX;                                ▷ saturar
7 fin
8 devolver  $\delta$  como u32;
```

Complejidad.

Tiempo y espacio constantes. El almacenamiento por arista es de 32 bytes, dando $\Theta(32m)$ para la lista de aristas de un grafo con m aristas.

3.3. ¿Por qué no `\#[repr(C, packed)]`?

Un layout empaquetado ahorraría otros 5 bytes eliminando el pad final, llegando a 27 bytes por arista ($\sim 16\%$ adicional). El motor no lo usa porque:

1. Cada lectura de un campo empaquetado requiere una copia byte a byte hacia storage alineado para satisfacer el invariante de alineación de referencias en Rust. El matcher DFS lee `source\_sid` y `dest\_sid` millones de veces por hunt; ese overhead de copia dominaría cualquier ahorro de memoria.
2. La alineación a 32 bytes coincide con la unidad de load AVX2, así que los barridos SIMD de la lista de aristas se mantienen amigables a la línea de caché.

Por eso el pad final queda como espacio futuro para más extensiones MVCC (p. ej. un puntaje de confianza o un bit flag de tag de fuente).

En el código.

`graph\_hunter\_core/src/relation.rs:83` — definición del struct.
`graph\_hunter\_core/src/relation.rs:151` — helper `encode\_ingested\_at`.
`graph\_hunter\_core/tests/mvcc\_ingested\_at.rs` — tests de round-trip MVCC.
Tests de módulo en `graph\_hunter\_core/src/relation.rs` — invariante de 32 bytes.

Capítulo 4

El internador de strings

Los identificadores de entidad en los logs ingeridos son largos y enormemente repetitivos — una sola tabla `SecurityEvent` en Sentinel menciona el mismo nombre de host cientos de miles de veces. Guardarlos como `String` propios dentro de `CompactRelation` haría estallar la lista de aristas. El motor esquiva esto con un internador bidireccional que mapea cada string único a un handle de 32 bits `StrId`.

4.1. Estructura

```
pub struct StrId(u32);

pub struct StringInterner {
    map:      HashMap<Arc<str>, StrId>, // lookup inverso
    strings: Vec<Arc<str>>,           // lookup por indice
}
```

Ambos contenedores guardan `Arc<str>` apuntando al mismo slice en heap, de modo que cada intern cuesta exactamente una alocaión (el `Arc::from(&str)` del camino `intern`) más dos incrementos de refcount. La alternativa — `String` en el map, `String` o una segunda copia en el vector — duplicaría la presión al allocator.

Invariante (Biyección del internador). Para todo índice $i \in [0, \text{strings.len}())$ se cumple que `map[strings[i]] == StrId(i as u32)`. Ambos contenedores se llenan atómicamente dentro de `intern`; un panic en medio de la inserción no puede dejarlos desincronizados porque una entrada a medio insertar no es visible a búsquedas posteriores (el `map.insert` es la última acción del happy path).

4.2. Algoritmos

Complejidad.

INTERN es $O(1)$ amortizado; el `HashMap` usa ahash, que pega muy pocas colisiones sobre IDs reales de entidad, y el `Vec::push` amortiza. La alocaión de heap ocurre sólo en el camino de miss. RESOLVE es $\Theta(1)$: un bounds check y una desreferencia de puntero. El consumo de espacio es

$$\Theta\left(\sum_{s \in \mathcal{U}} |s| + \Theta(|\mathcal{U}|)\right)$$

donde $\mathcal{U}$ es el conjunto de strings únicos — el primer término es el contenido heap, el segundo es el overhead $O(1)$ por entrada de los dos contenedores.

Algorithm 4.1: INTERN

Entrada: internador I , string s
Salida: StrId asignado a s

```

1 si  $s \in I.map$  entonces
2 |   devolver  $I.map[s]$ ;
3 fin
4  $id \leftarrow I.strings.len()$  como u32;
5  $arc \leftarrow Arc::from(s)$ ;                                ▷ 1 alloc heap
6  $I.strings.push(arc.clone())$ ;                                ▷ refcount++
7  $I.map.insert(arc, StrId(id))$ ;                                ▷ refcount++
8 devolver StrId(id);

```

Algorithm 4.2: RESOLVE

Entrada: internador I , StrId x
Salida: &str para x , o $\perp$ si está fuera de rango

```

1 si  $x.0 \geq I.strings.len()$  entonces
2 |   devolver  $\perp$ ;
3 fin
4 devolver & *  $I.strings[x.0]$ ;

```

4.3. Métricas

El motor expone el uso de memoria del internador vía `StringInterner::metrics()`:

```

pub struct InternerMetrics {
    pub unique_strings: usize,
    pub approx_bytes:   usize,    // suma de Arc<str>.len()
}

```

Éstas son las métricas gatillo para los backends diferidos *generacional* o *particionado* (Capítulo 23): cuando `unique\_strings` cruza $\sim 10^7$, o cuando una sesión *streaming* de larga vida crece monotónicamente por encima de unos cientos de MB en `approx\_bytes`, el operador sabe que es momento de cambiar.

4.4. El punto de costura (*seam*) de trait

El refactor P2-F introdujo `StringInternerBackend` como punto de intercambio:

```

pub trait StringInternerBackend {
    fn intern(&mut self, s: &str) -> StrId;
    fn resolve(&self, id: StrId) -> &str;
    fn try_resolve(&self, id: StrId) -> Option<&str>;
    fn get(&self, s: &str) -> Option<StrId>;
    fn len(&self) -> usize;
    fn is_empty(&self) -> bool;
    fn reserve(&mut self, additional: usize);
    fn metrics(&self) -> InternerMetrics;
}

```

`StringInterner` es la única implementación hoy. Cuando el motor reúna evidencia de que la hipótesis del internador global se quiebra (proceso multi-tenant, streaming 24×7 con más de 10^7 strings únicos), un `GenerationalInterner` o `PartitionedInterner` encaja detrás del mismo

trait; los 121+ call sites existentes siguen compilando porque mantienen un `StringInterner` concreto y el `metrics()` inherent delega en el cuerpo del trait.

En el código.

`graph\_hunter\_core/src/interner.rs:1` — struct, trait y métricas.
`graph\_hunter\_core/src/interner.rs:115` — tests unitarios que fijan la biyección.

Capítulo 5

El grafo en memoria

El struct central es `InMemoryGraph` (con alias `GraphHunter` por compatibilidad). Es el único productor de `CompactRelation`, el único dueño del `StringInterner` y del `MetadataStore`, y el único target del matcher DFS.

5.1. Campos

```
pub struct InMemoryGraph {
    // Almacenamiento directo.
    pub(crate) entities:      HashMap<StrId, Entity>,
    pub(crate) edge_store:    SpillableEdgeStore,
    pub(crate) interner:      StringInterner,
    pub(crate) meta_store:    MetadataStore,

    // Indices -- desnormalizados para velocidad de consulta.
    pub(crate) reverse_adj:   HashMap<StrId, Vec<StrId>>,
    pub(crate) type_index:    HashMap<EntityType, HashSet<StrId>>,
    pub(crate) entity_type_tags: Vec<u8>, // denso por StrId

    // Internador de datasets (Arc<str>, tag 16-bit).
    pub(crate) dataset_tags:  Vec<Arc<str>>,
    pub(crate) dataset_map:   HashMap<Arc<str>, u16>,
}
```

La redundancia entre `entities[...].entity\_type` y `entity\_type\_tags[...]` es deliberada: el primero es la fuente canónica pero cuesta un lookup en `HashMap`; el segundo es un array SoA indexado por `StrId.0` as `usize` — una lectura de byte en tiempo constante que el matcher usa dentro de su loop inner para descartar aristas cuyo tipo de destino no matchea el paso de la hipótesis.

Invariante (Coherencia de índices). Para todo `StrId x` producido por `interner.intern(\_)`: `entities.contains\_key(\&x)` si y sólo si el string fue agregado como entidad, y en ese caso `type\_index[et].contains(\&x)` con `et = entities[\&x].entity\_type`, y `entity\_type\_tags[x.0 as usize]` es igual al tag numérico de ese tipo de entidad.

5.2. Algoritmos de inserción

Complejidad.

ADD-ENTITY: $O(1)$ amortizado en la cantidad de entidades existentes; el `resize` de `entity\_type\_tags` se amortiza con la estrategia de duplicación de vectores. ADD-RELATION: $O(1)$ amortizado, do-

Algorithm 5.1: ADD-ENTITY

Entrada: grafo G , entidad $e = (\text{id}, \text{type})$

```

1  $x \leftarrow G.\text{interner.intern}(e.\text{id});$ 
2 si  $x \in G.\text{entities}$  entonces
3   | devolver ; ▷ idempotente en dup
4 fin
5  $G.\text{entities.insert}(x, e);$ 
6  $G.\text{type\_index}[\text{e.type}].\text{insert}(x);$ 
7  $G.\text{entity\_type\_tags.resize}(\max(\text{len}, x.0+1), \_);$ 
8  $G.\text{entity\_type\_tags}[x.0] \$\text{gets} \$ \text{tag}(\text{e.type});$ 
9  $G.\text{reverse\_adj.entry}(x).\text{or\_default}();$ 
10 devolver;
```

Algorithm 5.2: ADD-RELATION

Entrada: grafo G , relación $r = (\text{src}, \text{dst}, \text{rel}, \tau, M)$

```

1  $s \leftarrow G.\text{interner.get}(r.\text{src});$  fallar si None;
2  $d \leftarrow G.\text{interner.get}(r.\text{dst});$  fallar si None;
3  $o \leftarrow G.\text{meta\_store.append}(M);$ 
4  $t \leftarrow \text{CODIFICAR-INGESTED-AT}(\text{NOW-SECONDS}());$ 
5  $c \leftarrow \text{CompactRelation}(s, d, \text{rel}, \tau, o, \text{dsTag}, t);$ 
6  $G.\text{edge\_store.append}(s, c);$ 
7  $G.\text{reverse\_adj}[d].\text{push}(s);$ 
8 devolver;
```

minado por el `append` del edge store spillable (Capítulo 6) y el lookup del internador. El espacio por arista insertada es 32 bytes más cualquier byte de metadato apendizado al `MetadataStore`.

5.3. Finalización

Las listas de adyacencia dentro de `edge\_store` no están ordenadas por timestamp al insertar — ordenar en cada ADD-RELATION pagaría $O(\log d)$ por inserción y la ingesta viene de fuentes streaming en bulk. En su lugar, tras un batch, el llamador invoca `sort\_edges\_by\_timestamp`, que es el disparador externo de `SpillableEdgeStore::finalize`:

Algorithm 5.3: SORT-EDGES-BY-TIMESTAMP

Entrada: grafo G

```

1  $G.\text{edge\_store.finalize}();$  ▷ merge k-way, §6
2 para  $s \in G.\text{entities.keys}$  hacer
3   | ordenar  $G.\text{edge\_store.get\_edges}(s)$  in place por  $(\text{timestamp}, \text{dest\_sid})$  ;
3   |   ▷  $O(d(s) \log d(s))$ 
4 fin
```

Complejidad.

$O(m \log \bar{d})$ donde $\bar{d}$ es el grado saliente promedio, ya que cada lista de adyacencia de largo $d(v)$ se ordena por separado. Si el store estaba en modo spill, `finalize` primero hace un merge k -way a costo $O(m \log k)$ (Capítulo 6). El espacio sólo crece con el tamaño del archivo de merge durante este paso.

En el código.

graph_hunter_core/src/graph.rs:67 — campos del struct.
graph_hunter_core/src/graph.rs:211 — ADD-ENTITY.
graph_hunter_core/src/graph.rs:227 — ADD-RELATION.
graph_hunter_core/src/graph.rs:437 — SORT-EDGES-BY-TIMESTAMP.

Capítulo 6

El Spillable Edge Store

Ingstar un dump EVTX de 28 GB en una laptop con 16 GB de RAM requiere volcar (*spill*) las estructuras en memoria a disco antes de que el proceso sea liquidado por OOM. `SpillableEdgeStore` lo implementa para la lista de aristas: las aristas se acumulan en un buffer en memoria hasta cruzar un umbral; pasado el umbral, el buffer se ordena y se escribe como un archivo “run” ordenado en disco, y se arranca un buffer nuevo. En query time, el store presenta una única lista de adyacencia lógica haciendo merge de todos los runs en `finalize`.

6.1. Modos

Definición 6.1 (Modo del store). El store está *en memoria* (sin runs en disco) o *spilled* (uno o más runs en disco). Un tercer estado “finalizado” se alcanza después de que `finalize` unificó todos los runs en un único archivo mmap; una vez finalizado, el store es de sólo lectura.

Campos:

```
struct SpillableEdgeStore {
    buffer:          Vec<CompactRelation>,
    buffer_index:    HashMap<StrId, Vec<u32>>, // posiciones en buffer
    runs:            Vec<SpillRunHandle>,      // runs ordenados en disco
    merged_mmap:     Option<MmapFile>,         // estado finalizado
    finalized:       bool,
    spill_limit:     usize,                    // aristas antes de flush
}
```

`buffer\_index[s]` guarda las *posiciones* (índices u32) de las aristas de la source `s` dentro del buffer actual, no las aristas mismas. Esto hace barato a `get\_edges(s)` en modo en memoria: el llamador obtiene un slice cortado por esas posiciones, sin una copia HashMap-of-Vecs.

6.2. Append

Algorithm 6.1: SPILL-APPEND

Entrada: store S , source s , compact c

- 1 **si** $S.buffer.len() \geq S.spill_limit$ **entonces**
- 2 | `FLUSH-BUFFER-TO-RUN( $S$ )`; ▷ ver abajo
- 3 **fin**
- 4 $p \leftarrow S.buffer.len()$;
- 5 $S.buffer.push(c)$;
- 6 $S.buffer_index[s].push(p)$;

Algorithm 6.2: FLUSH-BUFFER-TO-RUN**Entrada:** store S

- 1 ordenar $S.buffer$ lexicográficamente por $(source_sid, timestamp, dest_sid)$;
- 2 escribir el buffer ordenado en un archivo temp f como bytes crudos (con header que lleva count y versión);
- 3 $S.runs.push(handle(f))$;
- 4 vaciar $S.buffer$ y $S.buffer_index$;

Complejidad.

SPILL-APPEND es $O(1)$ amortizado. Cada `spill\_limit` appends dispara un FLUSH de costo $O(L \log L)$ donde $L = spill_limit$, distribuido a lo largo del stream de ingesta. El costo total en modo spill para ingerir N aristas es $O(N \log L + N/L \cdot L) = O(N \log L)$ — logarítmico en el tamaño del flush, lineal en N .

6.3. Finalización vía merge k -way

Después de terminar todos los appends, `finalize` produce un único stream ordenado en disco. Si no hubo flushes el camino es trivial: ordenar el buffer en memoria, listo. Si no, el motor abre cursores sobre cada run más un cursor sobre el buffer en memoria ordenado, y los mergea con un min-heap:

Algorithm 6.3: K-WAY-MERGE**Entrada:** cursores $[c_0, \dots, c_{k-1}]$, archivo de salida f

- 1 construir un min-heap binario H sobre los cursores, clavado en su elemento cabecera $(source_sid, timestamp, dest_sid)$;
- 2 **mientras** $H \neq \emptyset$ **hacer**
- 3 $c \leftarrow H.pop_min()$;
- 4 escribir el elemento cabecera de c en f ;
- 5 $c.advance()$;
- 6 **si** c no está vacío **entonces**
- 7 $H.push(c)$
- 8 **fin**
- 9 **fin**

Complejidad.

Cada uno de los N elementos se popea y pushea exactamente una vez del heap, pagando $O(\log k)$ por operación, con total $O(N \log k)$ en tiempo. El heap tiene a lo sumo k cursores en cualquier instante, así que el uso de RAM es $O(k)$ independiente de N — esta es la propiedad que permite al motor procesar un stream de 28 GB en 16 GB de RAM. El output mergeado se mapea en memoria (mmap) para lecturas posteriores, así que `get\_edges(s)` sigue siendo $O(d(s))$.

Teorema 6.2 (Cota de memoria del spill). *En cualquier momento durante la ingesta el footprint en memoria de `SpillableEdgeStore` es a lo sumo $spill_limit \cdot size_of::<CompactRelation>$ más unos pocos KB de bookkeeping.*

Demostración. SPILL-APPEND flushes el buffer cuando su longitud alcanza `spill\_limit`; entre flushes el buffer guarda a lo sumo `spill\_limit` elementos, cada uno de 32 B, más el índice `HashMap<StrId, Vec<u32>>` que suma $O(L)$ entradas. Los runs en disco no ocupan RAM hasta mergearse. El merge propiamente dicho sólo mantiene k cursores en el heap, acotados por la cantidad de flushes; cada cursor pre-bufferea un bloque IO chico (unos KB). □ □

6.4. Lecturas

Algorithm 6.4: GET-EDGES

Entrada: store S , source s
Salida: slice de `CompactRelation` para s , vacío si desconocido

```

1 si  $S.finalized$  and  $S.merged\_mmap$  es Some entonces
2   |   localizar el offset inicial de  $s$  vía el índice en disco ;           ▷ bsearch  $O(\log n)$ 
3   |   devolver slice de largo count desde el mmap;
4 fin
5 si  $\neg S.finalized$  entonces
6   |   devolver posiciones de S.buffer\_index[$s$] materializadas desde S.buffer ;
        ▷  $O(d(s))$ 
7 fin
```

Complejidad.

$O(\log n + d(s))$ en el caso finalizado, $O(d(s))$ en el no finalizado (más el trabajo de copiar posiciones en un slice).

En el código.

`graph\_hunter\_core/src/spill.rs` — struct y todos los algoritmos.
`graph\_hunter\_core/src/spill.rs:327` — entry point de `finalize`.
`graph\_hunter\_core/src/spill.rs:524` — `get\_edges`.
`graph\_hunter\_core/benches/dedup\_throughput.rs` — bench de ingesta con spill.

Capítulo 7

El Metadata Store

Los metadatos por arista (pares clave-valor arbitrarios en `HashMap<String, String>` que los parsers llenan) son el campo de tamaño más variable asociado a una relación. Inlinearlos dentro de `CompactRelation` forzaría un registro de largo variable y destruiría el invariante de 32 bytes (Capítulo 3). El motor en cambio guarda los metadatos externamente en `MetadataStore` y deja sólo un offset de 64 bits en `CompactRelation`.

7.1. Estructura

`MetadataStore` es un buffer de bytes append-only con una codificación simple prefijada por longitud:

```
pub struct MetadataStore {
    buf: Vec<u8>,    // append-only
}
// layout de un record en offset o:
//   u32 num_entries
//   num_entries veces:
//     u16 key_len, bytes de la key, u16 value_len, bytes del value
```

El primer offset de record válido es 1, así que `metadata\_offset = 0` en `CompactRelation` significa inequívocamente “sin metadatos”. En el `append`, el store devuelve el offset donde escribió el record nuevo y ese offset vive en `CompactRelation` el resto de la sesión.

7.2. Operaciones

Algorithm 7.1: METADATA-APPEND

Entrada: store M , mapa $K : \{(k, v)\}$
Salida: offset $o \geq 1$, o 0 si K está vacío

- 1 **si** K está vacío **entonces**
- 2 | devolver 0;
- 3 **fin**
- 4 $o \leftarrow M.\text{buf}.\text{len}() + 1$;
- 5 escribir `u32(len($K$))` y luego cada par (`key`, `value`) con prefijos de largo `u16` en $M.\text{buf}$;
- 6 **devolver** o ;

Algorithm 7.2: METADATA-READ

Entrada: store M , offset o
Salida: $\text{HashMap}\langle \text{String}, \text{String} \rangle$, o vacío si $o = 0$

```

1 si  $o = 0$  entonces
2   | devolver mapa vacío;
3 fin
4 posicionar cursor en  $o - 1$  dentro de  $M.\text{buf}$ ;
5 leer u32  $\text{num}$ ;
6 para  $i \leftarrow 0$  a  $\text{num} - 1$  hacer
7   | leer u16  $k\_len$ , siguientes  $k\_len$  bytes como UTF-8  $\rightarrow k$ ;
8   | leer u16  $v\_len$ , siguientes  $v\_len$  bytes como UTF-8  $\rightarrow v$ ;
9   | insertar  $(k, v)$ ;
10 fin
11 devolver mapa;
```

Complejidad.

METADATA-APPEND: $O(|K| + \sum |k| + \sum |v|)$. METADATA-READ: lo mismo. El espacio por arista es cero si el parser no produjo metadatos, si no la longitud en bytes del record codificado. Una arista Sysmon típica produce ~ 60 bytes de metadatos, comparado con los 32 bytes de `CompactRelation` — por eso importa el store externo: las aristas se mantienen densas en el hot path del matcher y los metadatos sólo se pagan cuando un analista los pide.

7.3. Inmutabilidad

Invariante (Metadata append-only). Una vez que un record fue escrito en `MetadataStore.buf`, ninguna operación trunca o reescribe los bytes previos. Los offsets son entonces estables por la vida del grafo. En particular, el matcher puede capturar un offset dentro de un `CompactRelation` en tiempo de ingesta y leer los metadatos a través de él en tiempo de hunt sin preocuparse por mutaciones concurrentes del writer.

Cuando se persiste una sesión el `MetadataStore.buf` entero se serializa junto con las listas de entidades/relaciones; al cargar, el store se reconstituye verbatim preservando todos los offsets.

En el código.

`graph\_hunter\_core/src/metadata\_store.rs` — struct y rutinas de append/read.
`graph\_hunter\_core/src/graph.rs:246` — donde `ADD-RELATION` llama a `meta\_store.append`.

Capítulo 8

El AST de hipótesis

Cada hunt es guiado por un valor `Hypothesis` parseado. El AST es deliberadamente chico — un vector plano de pasos con listas de predicados por paso, más cláusulas opcionales de agregación y ventana temporal adosadas a la cadena completa.

8.1. Tipos

```
pub struct Hypothesis {
    pub name:      String,
    pub steps:      Vec<HypothesisStep>,
    pub k_simplicity: usize,           // default 1
    pub aggregations: Vec<AggClause>,  // cláusulas HAVING
    pub window_seconds: Option<u64>,  // cláusula WITHIN
}

pub struct HypothesisStep {
    pub origin_type:      EntityType,
    pub relation_type:     RelationType,
    pub dest_type:        EntityType,
    pub edge_predicates:   Vec<Predicate>, // sobre metadata de arista
    pub dest_predicates:   Vec<Predicate>, // sobre metadata de destino
    pub origin_binding:    Option<String>, // '(User u)' para HAVING
    pub dest_binding:      Option<String>,
}

pub enum Predicate {
    Eq      { key: String, value: String },
    Substr   { key: String, pattern: String },
    Regex    { key: String, pattern: String },
    In       { key: String, values: Vec<String> },
}

pub struct AggClause {
    pub op:      AggOp,           // Count, CountDistinct, Min, Max, Sum,
                                Avg
    pub column:  AggColumn,       // nombre binding o índice de paso
    pub cmp:     AggComparison,   // >=, <=, >, <, ==
    pub value:   f64,
}
```

8.2. Invariantes

Invariante (Cantidad de pasos ≥ 1). Una hipótesis debe tener al menos un paso; el parser del DSL rechaza input vacío con `ApiError::InvalidInput`. Esto libera al matcher de tratar el caso de cero aristas como una rama especial.

Invariante (k -simplicidad positiva). $k_{\text{simplicity}} \geq 1$. Cero admitiría sólo el camino vacío y violaría el sentido de “máximo de visitas por vértice”.

Invariante (Unicidad de bindings). Si `origin\_binding` o `dest\_binding` es `Some(b)` en algún paso, b no se repite en ninguna otra posición de binding dentro de la hipótesis. El parser lo hace cumplir en parse time así el matcher puede resolver bindings por nombre durante la agregación sin desambiguación.

8.3. Chequeo estático de compatibilidad

Antes de correr un hunt, el motor puede cruzar la hipótesis contra el schema de relaciones del grafo para ver si algún paso es estructuralmente imposible sobre el dataset cargado — p. ej. un paso `User -[Auth]-> Host` contra un grafo puramente de identidad cloud donde no existen entidades `Host`. Esto es `CHECK-CATALOG-COMPATIBILITY` (Capítulo 25), que devuelve un `CatalogEntryWithStatus` marcando los tipos de entidad o relación faltantes por paso.

Algorithm 8.1: CHECK-STEP-COMPATIBILITY

Entrada: schema Σ (multiconjunto de triples (source, rel, target)), paso s
Salida: bool compatible, conjunto de ejes faltantes

```

1 para cada  $(src, r, dst) \in \Sigma$  hacer
2   si  $(src = s.origin\_type \vee s.origin\_type = *)$ 
    $\wedge (r = s.relation\_type \vee s.relation\_type = *) \wedge (dst = s.dest\_type \vee s.dest\_type = *)$ 
   entonces
3     devolver (true,  $\emptyset$ );
4   fin
5 fin
6 faltantes  $\leftarrow$  conjunto de ejes ausentes en todo triple de  $\Sigma$ ;
7 devolver (false, faltantes);
```

Complejidad.

$O(|\Sigma|)$ por paso, $O(|\Sigma| \cdot \ell)$ por hipótesis, donde ℓ es la cantidad de pasos. $|\Sigma|$ está acotado en la práctica por $|\mathcal{T}_V|^2 \cdot |\mathcal{T}_E|$ (hay pocas decenas de tipos de entidad/relación), así que el chequeo es efectivamente constante.

En el código.

```

graph\_hunter\_core/src/hypothesis.rs — Hypothesis, HypothesisStep, Predicate.
graph\_hunter\_core/src/catalog/mod.rs:105 — check\_catalog\_compatibility.
```

Parte III

Algoritmos principales

Capítulo 9

Parseo del DSL

El DSL es una pequeña gramática de cadena de flechas diseñada para leerse en voz alta (“Un User se autentica a un Host y ejecuta un Process”) y para entrar en una línea de input de UI. El parser es un descendiente recursivo escrito a mano con una sola pasada — sin archivo de gramática, sin dependencia de generador.

9.1. Gramática

```
hypothesis      := step ( step )* modifier*
step             := entity_type arrow
arrow           := "-[" rel_type predicates? "]"->" entity_type binding?
                  predicates? dest_predicates?
entity_type     := ident ( "(" binding ")" )? | "*"
rel_type        := ident | "*"
predicates      := "{" predicate ( "," predicate )* "}"
predicate       := key ("=" | "~" | "in ") (literal | list)
modifier        := having | within | k_clause
having          := "HAVING" agg_op "(" agg_arg ")" cmp literal
within          := "WITHIN" int time_unit
k_clause        := "{k=" int "}"
```

Cada tipo de token se reconoce por una prueba de clase de caracter, de modo que el parser hace exactamente una pasada sobre el string de input. No hay backtracking; la gramática es LL(1) tras el lookahead a nivel lexer para predicados.

9.2. Algoritmo

Complejidad.

$O(|src|)$ en tiempo y espacio. Cada token se visita una sola vez; las listas de predicados basadas en `HashMap` son $O(1)$ para poblar. El espacio es lineal en la longitud de la cadena parseada más los literales de predicados.

Teorema 9.1 (Determinismo del parser). *PARSE-DSL es determinista: para todo input string fijo, produce el mismo `Hypothesis` o el mismo `DslError` en cada invocación.*

Demostración. No hay orden de iteración de hash-map, no hay valores aleatorios, no hay llamadas al reloj del sistema en ningún lugar del parser. Todas las estructuras construidas durante el parseo (las `edge\_predicates` y `dest\_predicates` finales en cada paso) preservan el orden de inserción. □ □

Algorithm 9.1: PARSE-DSL

Entrada: string `src`, nombre opcional de hipótesis `nm`**Salida:** Hypothesis o DslError

```

1 toks ← TOKENIZE(src) ; ▷  $O(|\text{src}|)$ 
2 steps ←  $\emptyset$ ;
3  $\sigma_0 \leftarrow \text{PARSE-ENTITY-TYPE}(\text{toks})$ ;
4 mientras toks tiene “→” restante hacer
5   |  $\rho, P^{\text{edge}} \leftarrow \text{PARSE-EDGE-SPEC}(\text{toks})$ ;
6   |  $\sigma_i \leftarrow \text{PARSE-ENTITY-TYPE}(\text{toks})$ ;
7   |  $P^{\text{dst}} \leftarrow \text{PARSE-DEST-PREDICATES}(\text{toks})$ ;
8   | empujar HypothesisStep( $\sigma_{i-1}, \rho, \sigma_i, P^{\text{edge}}, P^{\text{dst}}$ ) a steps;
9 fin
10  $k \leftarrow \text{parsear } \{\text{k=N}\}$  opcional, default 1;
11  $A \leftarrow \text{PARSE-HAVING}(\text{toks})$ ;
12  $w \leftarrow \text{PARSE-WITHIN}(\text{toks})$ ;
13 devolver Hypothesis(nm, steps,  $k$ ,  $A$ ,  $w$ );
```

En el código.

graph_hunter_core/src/dsl.rs:47 — entry point parse_dsl.

graph_hunter_core/src/dsl.rs — implementación de gramática completa (helpers privados).

Capítulo 10

Matcheo de patrones temporales

El matcher es el hot path del motor. Cada hunt que corre el analista, cada llamada `run\_hunt` desde Claude, cada invocación CLI termina acá. La implementación trae tres variantes coordinadas:

- SEARCH-TEMPORAL-PATTERN — el entry point; despacha trabajo a workers rayon y junta resultados.
- DFS-MATCH-ITERATIVE — el DFS iterativo base que se ejecuta por vértice de partida.
- DFS-MATCH-SMART — variante con cota de anomalía que mantiene un min-heap top- K y poda ramas cuyo estimado corriente las hace inadmisibles.

Un cuarto camino, SEARCH-TEMPORAL-PATTERN-SIMD, delega en `libgraphmatch` o en el módulo puro-Rust `simd\_rust` (Capítulo 11) y está gateado por el feature `simd` más una prueba de CPU en runtime.

10.1. Estado compartido

```
// Scratch alocado una vez por worker y reusado entre starts.
struct DfsFrame {
    sid:          StrId,
    step_idx:     usize,
    edge_hi:      usize,    // cota superior de aristas salientes
    edge_cursor:  usize,    // siguiente indice de arista a probar
}

struct SmartDfsFrame {
    sid:          StrId,
    step_idx:     usize,
    edge_hi:      usize,
    edge_cursor:  usize,
    path_anomaly_sum: f64,    // suma de node_anomaly_estimate a lo largo
                             del path
}
```

Los frames de la pila son structs comunes guardados en un `Vec<Frame>` que se reusa entre vértices de partida. Este es el ahorro de alocaión más grande del matcher — la implementación recursiva anterior alocaba por frame en la pila del sistema y no podía reiniciarse trivialmente.

Invariante (k -simplicidad). El `visit\_count: HashMap<StrId, usize>` corriente mantenido dentro de cada caminata DFS satisface `visit\_count[v] <= k` para todo v en el camino actual, donde k es `hypothesis.k\_simplicity`. Violarlo admitiría caminos que revisitan un vértice más

de k veces, contradiciendo la semántica de la hipótesis. El algoritmo impone el invariante en el push: si incrementar `visit\_count[dest]` excedería k , la arista se descarta.

10.2. Entry point: Search-Temporal-Pattern

El algoritmo externo particiona el trabajo por vértice de partida. Por cada entidad cuyo tipo matchea el primer paso de H , un thread rayon ejecuta DFS-MATCH-ITERATIVE sobre ese vértice con su propia pila scratch y su propio buffer de resultados. Cuando todos los workers terminan, los resultados se mergean, puntúan, deduplican y paginan (Capítulo 14).

Algorithm 10.1: SEARCH-TEMPORAL-PATTERN

Entrada: grafo G , hipótesis H , ventana opcional (t_0, t_1) , cap opcional max
Salida: vector de vectores de caminos

```

1 starts  $\leftarrow$  G.type\_index[H.steps[0].origin\_type];
2 si starts =  $\emptyset$  entonces
3   | devolver  $\emptyset$ ;
4 fin
5 cap  $\leftarrow$  AtomicUsize(0);
6 results  $\leftarrow$  map paralelo sobre starts: para cada  $s \in$  starts en paralelo hacer
7   | local  $\leftarrow$  DFS-MATCH-ITERATIVE(G, H, s, t_0, t_1, cap, max);
8   | devolver local;
9 fin
10 devolver concat de results;
```

Cada worker lee el `AtomicUsize` cap en cada push de candidato, de modo que cuando el conteo agregado alcanza max, los workers restantes paran lo antes posible. Es un patrón clásico de early-exit: el count puede rebasar max momentáneamente porque los workers *commit* y luego chequean, pero el overshoot está acotado por la cantidad de workers rayon (típicamente 8–16).

10.3. El DFS iterativo

PUSH-EDGES-FOR-STEP es una rutina de una línea que hace binary search en `edge\_store[v]` para la primera arista con `timestamp` $\geq t_0$ y setea `edge\_hi` a la primera arista con `timestamp` $> t_1$. El sort por timestamp que se hace en finalize (Capítulo 5) es lo que justifica que la binary search funcione.

Teorema 10.1 (Corrección del matcher). *Al retornar, DFS-MATCH-ITERATIVE produce exactamente el conjunto de caminos temporales de longitud ℓ rooteados en s_0 que satisfacen la hipótesis H dentro de $[t_0, t_1]$ y respetan la k -simplicidad.*

Demostración (esbozo). Por inducción sobre la profundidad de la hipótesis. El caso base ($\ell = 1$) es el primer push en `stack`; el chequeo es paso a paso equivalente a las restricciones de la hipótesis sobre la primera arista. El paso inductivo observa que cada arista admitida extiende el camino exactamente un vértice, actualiza `visit` consistentemente, y que el chequeo de ciclos rechaza exactamente aquellas extensiones que excederían k -simplicidad. La pila se desenrolla restaurando `visit` y el camino, de modo que cada rama se explora independientemente; ningún camino se omite por backtracking faltante. □ □

Complejidad.

El peor caso es un grafo donde toda arista matchea todo paso. Entonces desde un start s_0 el DFS visita hasta $\bar{b}^\ell$ caminos (con $\bar{b}$ = grado saliente promedio de aristas calificantes por paso y ℓ

Algorithm 10.2: DFS-MATCH-ITERATIVE

Entrada: grafo G , hipótesis H , start s_0 , ventana (t_0, t_1) , contador compartido cap, cota max

Salida: lista de caminos matcheantes roteados en s_0

```

1 results  $\leftarrow \emptyset$ ; stack  $\leftarrow \emptyset$ ; path  $\leftarrow [s_0]$ ; visit  $\leftarrow \{s_0 : 1\}$ ;
2 PUSH-EDGES-FOR-STEP(stack,  $G$ ,  $s_0$ , paso 0,  $t_0, t_1$ );
3 mientras stack  $\neq \emptyset$  hacer
4    $f \leftarrow$  tope de stack;
5   si  $f.\text{edge\_cursor} \geq f.\text{edge\_hi}$  entonces
6     | popear  $f$ ; popear último vértice de path; decrementar visit[·]; continuar;
7   fin
8    $c \leftarrow G.\text{edge\_store}[f.\text{sid}][f.\text{edge\_cursor}]$ ;
9    $f.\text{edge\_cursor} \leftarrow f.\text{edge\_cursor} + 1$ ;
10  // filtro: tipo de relación
11  si  $c.\text{rel\_type\_tag} \neq \text{paso}[f.\text{step\_idx}].\text{rel}$  entonces
12    | continuar;
13  fin
14  // filtro: tipo de entidad destino vía tag SoA
15   $u \leftarrow c.\text{dest\_sid}$ ;
16  si  $G.\text{entity\_type\_tags}[u.0] \neq \text{paso}[f.\text{step\_idx}].\text{dst}$  entonces
17    | continuar;
18  fin
19  // filtro: ventana temporal
20  si  $c.\text{timestamp} \notin [t_0, t_1]$  entonces
21    | continuar;
22  fin
23  // filtro: predicados de arista (sobre metadata)
24  si  $\text{EVALUATE-PREDICATES}(\text{paso}.edge\_predicates, c, G) = \text{falso}$  entonces
25    | continuar;
26  fin
27  // prevención de ciclos vía  $k$ -simplicidad
28  si  $\text{visit}[u] + 1 > H.k$  entonces
29    | continuar;
30  fin
31   $\text{visit}[u] \leftarrow \text{visit}[u] + 1$ ; anexar  $u$  a path;
32  si  $f.\text{step\_idx} + 1 = \text{len}(H.steps)$  entonces
33    | registrar path en results;
34    | incrementar cap atómicamente; si  $\text{cap} \geq \text{max}$  entonces
35    | | salir del loop;
36    | fin
37    |  $\text{visit}[u] \leftarrow \text{visit}[u] - 1$ ; popear último vértice; continuar;
38  fin
39  empujar frame nuevo:  $(u, f.\text{step\_idx} + 1, \dots)$  sobre stack;
40  PUSH-EDGES-FOR-STEP(stack,  $G$ ,  $u$ ,  $f.\text{step\_idx} + 1$ ,  $t_0, t_1$ );
41 fin
42 devolver results;

```

= cantidad de pasos), dando $O(\bar{b}^\ell \cdot \ell)$ por start. Cruzando los n_s candidatos de partida, el total es

$$T_{\text{match}}(G, H) = O(n_s \cdot \bar{b}^\ell \cdot \ell).$$

El espacio por worker es $O(\ell)$ para la pila y el camino más $O(\ell \cdot k)$ para el mapa `visit` (cada entrada es un vértice, y el camino tiene a lo sumo ℓ vértices distintos cada uno visitado a lo sumo k veces). Con W workers, el espacio total del matcher es $O(W \cdot \ell \cdot k)$. En la práctica $\bar{b}$ es diminuto porque el fast-path por tag de tipo elimina la mayoría de las aristas salientes en la primera comparación de byte; mediciones empíricas desde `benches/hunt\_latency.rs` están en el Ap. C.

10.4. La variante smart

DFS-MATCH-SMART se usa cuando el llamador pide los top- K caminos más anómalos en vez de “todos los caminos”. Difiere del iterativo sólo en el orden de candidatos y el acumulador de resultados:

- **Acumulador:** un min-heap de tamaño a lo sumo K ordenado por puntaje compuesto de anomalía. Cuando está lleno, los caminos nuevos se aceptan si y sólo si puntúan más alto que el mínimo del heap.
- **Cota corriente:** cada frame trackea `path\_anomaly\_sum`, la suma de `node\_anomaly\_estimate` (Capítulo 12) a lo largo del camino hasta el momento. Extender el camino no puede decrecer esta suma; si la suma ya cae por debajo del mínimo del heap, la rama se poda.

Algorithm 10.3: MERGE-INTO-TOPK

Entrada: heap local L , heap global G , capacidad K

```

1 para cada  $p \in L$  hacer
2   si  $|G| < K$  entonces
3      $G.\text{push}(p)$ 
4   fin
5   sino si  $\text{score}(p) > \text{mín}(G)$  entonces
6      $G.\text{pop\_min}(); G.\text{push}(p);$ 
7   fin
8 fin
```

Complejidad.

$O((|L| + |G|) \log K)$. Cuando el matcher junta los workers rayon, cada heap local de tamaño $\leq K$ se mergea en un único heap global a costo total $O(WK \log K)$, despreciable frente al trabajo del DFS.

10.5. Paralelismo

El matcher es embarazosamente paralelo en el vértice de partida. Como cada DFS de worker lee el grafo exclusivamente (todos los escritores están bloqueados en un `RwLock::write` a nivel sesión), no hay sincronización entre workers más allá del contador compartido `cap` y la reducción final en un único vector de resultados.

Teorema 10.2 (Speedup paralelo). *Dados W workers rayon y n_s vértices de partida con carga de DFS aproximadamente igual por start, el matcher alcanza un speedup de $\text{mín}(W, n_s)$ hasta el costo constante de la reducción final.*

La prueba es el teorema de Brent aplicado a la partición trivial: el DFS de cada start no tiene dependencias entre starts, y la reducción final es $O(\text{total resultados})$, típicamente órdenes de magnitud menor que el DFS en sí.

En el código.

```
graph\_hunter\_core/src/graph.rs:461 — search\_temporal\_pattern.  
graph\_hunter\_core/src/graph.rs:1121 — dfs\_match\_iterative.  
graph\_hunter\_core/src/graph.rs:782 — search\_temporal\_pattern\_smart.  
graph\_hunter\_core/src/graph.rs:932 — dfs\_match\_smart.  
graph\_hunter\_core/benches/hunt\_latency.rs — benchmarks de performance.
```

Capítulo 11

Aceleración SIMD

El trabajo inner del matcher DFS está dominado por el filtrado de aristas: chequear `rel\_type\_tag`, `entity\_type\_tags[dst]`, inclusión de timestamp y evaluación de predicados para millones de aristas candidatas por hunt. Tres de esos cuatro chequeos son comparaciones a nivel byte que vectorizan limpiamente. GraphHunter tiene dos caminos SIMD:

1. **libgraphmatch** — una biblioteca C++ enlazada por FFI cuando se compila el feature `simd` y el CPU host reporta soporte AVX2. El throughput peak es aproximadamente 3× el del matcher Rust escalar.
2. **simd_rust puro-Rust** (P2-D) — una implementación `std::arch::x86\_64` que usa intrínsecos AVX2 sin salir del toolchain Rust. Hoy implementa una sola primitiva, `INTERSECT-SORTED-U32`, con más en camino.

11.1. Puente FFI de libgraphmatch

```
// graph_hunter_core/src/simd_matcher.rs (abreviado)
unsafe extern "C" {
    fn gm_graph_new() -> *mut c_void;
    fn gm_graph_add_node(g: *mut c_void, id: u32, et: u8);
    fn gm_graph_add_edge(g: *mut c_void, src: u32, dst: u32, rt: u8, ts:
        i64);
    fn gm_graph_finalize(g: *mut c_void);
    fn gm_matcher_run(m: *mut c_void, max_results: u32, min_score: f64)
        -> *mut c_void;
    fn gm_results_free(r: *mut c_void);
    fn gm_graph_free(g: *mut c_void);
}
```

El lado Rust envuelve los handles `*mut c\_void` en `GmGraph` (RAII, drop llama a `gm\_graph\_free`) y cachea el grafo resultante junto con un `HashMap<String, u32>` que mapea string IDs a los índices de nodo C++ (ver `CachedSimdGraph` en `simd\_matcher.rs`). El caché sobrevive a los hunts en la misma sesión; se invalida al mutar el grafo.

SEARCH-TEMPORAL-PATTERN-SIMD es entonces una función corta:

El fallback al matcher smart en Rust es inevitable porque `libgraphmatch` no evalúa predicados de metadata (`\{key="val"\}`) — éstos requieren acceso al `MetadataStore`, que el lado C++ no puede hacer sin enviar los bytes crudos por FFI.

Algorithm 11.1: SEARCH-TEMPORAL-PATTERN-SIMD

Entrada: grafo G , hipótesis H , cap max

```

1 si AVX2 no detectado en runtime or  $H$  tiene predicados entonces
2 |   devolver SEARCH-TEMPORAL-PATTERN-SMART( $G, H, \dots$ ) ;           ▷ fallback
3 fin
4  $C \leftarrow G.\text{cached\_simd\_graph}()$  ;                               ▷ construir una vez, reusar
5  $r \leftarrow gm\_matcher\_run(\$C, \text{max}, 0.0)$ ;
6 devolver translate\_node\_ids( $r$ ,  $\$C.\text{text\_string\_map}$ );

```

11.2. Intersección AVX2 puro-Rust

El refactor P2-D empezó a reemplazar libgraphmatch una primitiva a la vez. La primera que se entregó es `simd\_rust::intersect\_sorted\_u32\_count`: dados dos slices `&[u32]` ascendentes deduplicados, retorna la cantidad de elementos en común. Esta es la primitiva que libgraphmatch usa para intersectar vecindades durante la expansión SIMD del patrón, y es la primitiva más barata de probar segura en aislamiento.

El algoritmo es el esquema clásico “broadcast-and-compare” usado en la literatura de intersección SIMD. Su corrección se desprende del invariante de que j sólo incrementa monótonicamente después de que la ventana completa $L[j..j+8]$ fue descartada (todos los elementos estrictamente menores que el s actual); como S y L están ordenados, ningún s posterior puede matchear en ningún índice $< j$.

Teorema 11.1 (Complejidad de la intersección). *Sobre inputs de tamaños $|A|$ y $|B|$ con $|S| = \min(|A|, |B|)$, $|L| = \max(|A|, |B|)$:*

- *Tiempo: $O(|S| \cdot (|L|/8 + 1))$ en el peor caso. El factor $/8$ es el ancho de lane AVX2. En la práctica la iteración por el lado corto amortiza cerca de $O(|S| + |L|/8)$ porque j avanza monótonicamente.*
- *Espacio: $O(1)$ auxiliar (el estado de registros).*

Teorema 11.2 (Seguridad en runtime). *INTERSECT-SORTED-U32-COUNT no invoca comportamiento indefinido aun cuando se compile sin AVX2, porque el cuerpo unsafe del intrínseco sólo se llama a través de `is\_x86\_feature\_detected!("avx2")` en runtime. El gate `#[target\_feature(enable = "avx2")]` en compile time sólo habilita el cuerpo del intrínseco dentro de una función que el path escalar no llama.*

Demostración (esbozo). El wrapper público es safe. Chequea `is\_x86\_feature\_detected!("avx2")` primero; sólo en hit invoca al cuerpo unsafe. Cuando AVX2 no se reporta, se llama al merge escalar y no se ejecutan instrucciones AVX2. El gate `target\_feature` significa que el cuerpo unsafe mismo se compila con codegen AVX2 de todos modos, pero eso está bien porque sólo se llama cuando el CPU lo soporta. □ □

11.3. Roadmap

Primitivas restantes por portar de libgraphmatch a `simd\_rust`:

- `count\_edges\_by\_rel\_type`: barrer un `&[CompactRelation]` buscando entradas que matcheen un `rel\_type\_tag` dado. Trivial (`_mm256_cmpeq_epi8` sobre el array de tags empaquetado).
- `timestamp\_range\_filter`: marcar aristas cuyo `timestamp` cae en $[t_0, t_1]$. Requiere comparaciones de 64 bits — una instrucción extra por lane.

Algorithm 11.2: INTERSECT-SORTED-U32-COUNT

Entrada: slices ordenados A, B de u32, ambos ascendentes y deduplicados**Salida:** cantidad de elementos comunes

```

1 si  $|A| < 32$  or  $|B| < 32$  entonces
2   | devolver SCALAR-MERGE( $A, B$ );
3 fin
4 si AVX2 no detectado en runtime entonces
5   | devolver SCALAR-MERGE( $A, B$ );
6 fin
  // lado corto es  $S$ , lado largo es  $L$ 
7 si  $|A| \leq |B|$  entonces  $(S, L) \leftarrow (A, B)$ ;
8 sino  $(S, L) \leftarrow (B, A)$ ;
9  $j \leftarrow 0$ ; count  $\leftarrow 0$ ;
10 para cada  $s \in S$  hacer
11   | mientras  $j + 8 \leq |L|$  hacer
12     | chunk  $\leftarrow$  AVX2 load  $L[j..j + 8]$ ;
13     | needle  $\leftarrow$  AVX2 broadcast  $s$  a 8 lanes;
14     | cmp  $\leftarrow$  \_mm256\_cmpeq\_epi32(chunk, needle);
15     | mask  $\leftarrow$  \_mm256\_movemask\_epi8(cmp);
16     | si mask  $\neq 0$  entonces
17       | count  $\leftarrow$  count + 1; romper loop interno;
18     fin
    // avanzar  $j$  si todos los lanes del chunk son  $< s$ 
19     | si  $L[j + 7] < s$  entonces
20       |  $j \leftarrow j + 8$ ; continuar;
21     fin
22   romper loop interno;
23 fin
24 SCAN-TAIL( $L, j, s$ , count) ; ▷ scan escalar para  $< 8$  restantes
25 fin
26 devolver count;
```

- `intersect\_sorted\_u32\_out`: la variante que materializa output de la primitiva actual sólo-count.

Cuando aterricen las tres, el path `libgraphmatch` de `SEARCH-TEMPORAL-PATTERN-SIMD` puede reemplazarse por un despacho puro-Rust, retirando la dependencia C++.

En el código.

`graph\_hunter\_core/src/simd\_matcher.rs` — puente FFI.
`graph\_hunter\_core/src/simd\_rust.rs` — `INTERSECT-SORTED-U32-COUNT`.
`libgraphmatch/src/simd\_intersect.cpp` — implementación C++ de referencia.
`graph\_hunter\_core/src/graph.rs:689` — `SEARCH-TEMPORAL-PATTERN-SIMD`.

Capítulo 12

Scoring de anomalía

El ranking de caminos es la segunda mitad de lo que hace el motor: tras matchear los caminos que satisfacen una hipótesis, el `AnomalyScorer` asigna a cada uno un puntaje compuesto en $[0, 1]$ y un desglose opcional por componente para que el analista pueda triagear. El scorer es un promedio ponderado de cinco dimensiones (rareza de entidad, rareza de arista, concentración de vecindad, novedad temporal y amenaza GNN), cada una computada como promedio sobre los vértices del camino.

12.1. Estado y finalización

```
pub struct AnomalyScorer {
    entity_freq:      HashMap<String, u64>,
    pair_freq:        HashMap<(String, String), u64>,
    first_seen:       HashMap<String, i64>,
    nc_cache:         HashMap<String, f64>,
    gnn_cache:        HashMap<String, f64>,
    log_max_freq:     f64,
    log_max_pair_freq: f64,
    tau_min:          i64,
    tau_max:          i64,
    weights:          ScoringWeights,
    finalized:        bool,
}
```

El scorer se puebla incrementalmente durante la ingesta (`observe\_entity`, `observe\_edge`); son $O(1)$ amortizado y no computan nada derivado. Después de la ingesta, `finalize` convierte los conteos crudos en los factores de normalización que el scoring por camino necesita.

Algorithm 12.1: ANOMALY-FINALIZE

Entrada: scorer S , grafo G

- 1 $S.\text{log_max_freq} \leftarrow \ln(\text{máx } \text{entity_freq.values})$;
- 2 $S.\text{log_max_pair_freq} \leftarrow \ln(\text{máx } \text{pair_freq.values})$;
- 3 $(\tau_{\text{mín}}, \tau_{\text{máx}}) \leftarrow (\text{mín}, \text{máx})$ de los valores no cero de `first\_seen.values`;
- 4 $S.\text{nc_cache} \leftarrow \emptyset$;
- 5 **para** cada $v \in G.\text{entities}$ **hacer**
- 6 $S.\text{nc_cache}[v] \leftarrow \text{COMPUTE-NC}(v, G)$;
- 7 **fin**
- 8 $S.\text{finalized} \leftarrow \text{verdadero}$;

Complejidad.

$O(V + E + \sum_v d(v)) = O(V + E)$ para los contadores. El cache de entropía de Shannon cuesta un adicional $O(\sum_v d(v)) = O(E)$. El espacio es $O(V)$ para los caches más lo que ya consumieron los contadores crudos.

12.2. Concentración de vecindad

Algorithm 12.2: COMPUTE-NC

Entrada: vértice v , grafo G
Salida: concentración de vecindad $\text{NC}(v) \in [0, 1]$

```

1 counts  $\leftarrow \emptyset$ ; total  $\leftarrow 0$ ;
2 para cada arista saliente  $(v, u) \in G$  hacer
3   | counts[ $\ell_V(u)$ ] += 1; total += 1;
4 fin
5 para cada arista entrante  $(w, v) \in G$  hacer
6   | counts[ $\ell_V(w)$ ] += 1; total += 1;
7 fin
8  $K \leftarrow |\text{counts}|$ ;
9 si  $K \leq 1$  entonces
10  | devolver 1.0;
11 fin
12  $H \leftarrow 0$ ;
13 para cada  $c \in \text{counts.values}$  hacer
14  |  $p \leftarrow c/\text{total}$ ;
15  |  $H \leftarrow H - p \log_2 p$ ;
16 fin
17  $H_{\text{máx}} \leftarrow \log_2 K$ ;  $\text{NC} \leftarrow 1 - H/H_{\text{máx}}$ ;
18 devolver clamp(NC, 0, 1);
```

NC es 1 cuando la vecindad es maximamente homogénea (una sola familia de tipo de entidad) y 0 cuando es maximamente diversa; el clamp protege contra *drift* de punto flotante cerca de los extremos.

Complejidad.

$O(d(v))$ en tiempo, $O(|\mathcal{T}_V|)$ en espacio (sólo conteos por tipo de vecino; constante chica).

12.3. Puntuar un camino

Tres de las cuatro componentes no-GNN son promedios simples sobre los vértices del camino; la componente de rareza de arista promedia sobre las aristas del camino.

Cada componente se clampa individualmente a $[0, 1]$ para que una entrada de cache desactualizada (extremadamente rara, pero posible durante un race de re-finalize) no pueda volver negativo a un componente. La renormalización de pesos significa que el llamador puede setearlos en cualquier escala absoluta.

Complejidad.

$O(\ell)$ en tiempo, $O(1)$ en espacio auxiliar. El estado caro lo construyó ANOMALY-FINALIZE; el scoring por camino es barato.

Algorithm 12.3: SCORE-PATH

Entrada: camino $\pi = (v_0, \dots, v_\ell)$, scorer S
Salida: puntaje compuesto c , desglose b

- 1 **si** π vacío **or** $\neg S.finalized$ **entonces**
- 2 | **devolver** $(0, 0)$;
- 3 **fin**
- // Componente 1: rareza de entidad
- 4 $ER \leftarrow \frac{1}{\ell+1} \sum_v \left(1 - \frac{\ln(\text{entity_freq}[v])}{S.\text{log_max_freq}}\right)$;
- // Componente 2: rareza de arista
- 5 $EdgeR \leftarrow \frac{1}{\ell} \sum_{i=1}^{\ell} \left(1 - \frac{\ln(\text{pair_freq}[v_{i-1}, v_i])}{S.\text{log_max_pair_freq}}\right)$;
- // Componente 3: promedio NC
- 6 $NC \leftarrow \frac{1}{\ell+1} \sum_v S.nc_cache[v]$;
- // Componente 4: novedad temporal
- 7 $TN \leftarrow \frac{1}{\ell+1} \sum_v \frac{\text{first_seen}[v] - \tau_{\min}}{\tau_{\max} - \tau_{\min}}$;
- // Componente 5: amenaza GNN
- 8 $GT \leftarrow \frac{1}{\ell+1} \sum_v S.gnn_cache.get(v).unwrap_or(0)$;
- // compuesto vía renormalización de pesos
- 9 $W \leftarrow \sum \text{weights}$; $c \leftarrow \frac{1}{W} \sum_i w_i \cdot \text{componente}_i$;
- 10 **devolver** $(\text{clamp}(c, 0, 1), \text{desglose})$;

12.4. El *seam* de trait (P2-A)

El módulo de scoring refactorizado (`graph\_hunter\_core/src/scoring/`) hace que cada componente sea una impl `ScoreComponent` separada:

```
pub trait ScoreComponent: Send + Sync {
    fn name(&self) -> &'static str;
    fn score_path(&self, ctx: &PathContext<'>) -> f64;
    fn explain(&self, value: f64, ctx: &PathContext<'>) -> Option<String>
        {
            None
        }
}
```

`CompositeScorer` sostiene un `Vec<Box<dyn ScoreComponent>>` más pesos coincidentes y aplica una agregación estilo SCORE-PATH. Agregar un sexto componente (bayesiano, modelo aprendido, etc.) es un struct nuevo más un `vec!{} push`.

Teorema 12.1 (Equivalencia v1/v2). *Para todo vector de pesos w y todo camino π , $AnomalyScorer::score_path_v1(w, \pi)$ y $AnomalyScorer::score_path_v2(w, \pi)$ coinciden dentro de $8 \cdot \epsilon_{f64}$.*

Esto lo verifica `tests/scoring\_snapshot.rs`. La equivalencia es la precondition para eventualmente retirar el v1 hardcodeado.

12.5. Estimación rápida a nivel nodo

Durante el podado del DFS smart (Capítulo 10) necesitamos una estimación $O(1)$ de la anomalía de un vértice solo para que la `path\_anomaly\_sum` corriente pueda rechazar ramas. Esa es `node\_anomaly\_estimate`:

```
pub fn node_anomaly_estimate(&self, v: &str) -> f64 {
    if !self.finalized { return 0.5; }
```

```

    let er = /* Componente 1 para un solo vertice */;
    let nc = self.nc_cache.get(v).copied().unwrap_or(1.0);
    let tn = /* Componente 4 para un solo vertice */;
    let gt = self.gnn_cache.get(v).copied().unwrap_or(0.0);
    combine_with_weights(er, nc, tn, gt, &self.weights)
}

```

La rareza de arista no es parte de la estimación porque el DFS todavía no eligió la arista siguiente cuando consulta la cota. La estimación es entonces una cota superior suelta de la contribución del nodo al compuesto; es admisible (nunca sobrepoda) siempre y cuando los pesos sean no-negativos, cosa que el motor impone al cargar la config.

En el código.

```

graph\_hunter\_core/src/anomaly.rs:148 — finalize.
graph\_hunter\_core/src/anomaly.rs:191 — compute\_nc.
graph\_hunter\_core/src/anomaly.rs:241 — score\_path (v1).
graph\_hunter\_core/src/anomaly.rs:351 — node\_anomaly\_estimate.
graph\_hunter\_core/src/scoring/ — v2 basado en trait.
graph\_hunter\_core/tests/scoring\_snapshot.rs — fijación de equivalencia v1/v2.

```

Capítulo 13

Centralidad y analíticos

Más allá de la ejecución de hunts, el motor ofrece un puñado de analíticos sobre el grafo: centralidad betweenness (para destacar nodos estructuralmente importantes), PageRank temporal (para identificar nodos cuya influencia llegó al pico cerca de un tiempo de referencia) y algunos helpers auxiliares como `get\_relation\_schema`. Este capítulo cubre los dos primeros en profundidad — son los analíticos no triviales y los que más probablemente sorprendan al lector con su costo.

13.1. Centralidad betweenness (Brandes con sampling)

El algoritmo de Brandes 2001 computa betweenness exacto en un grafo dirigido en $O(VE)$, lo cual es prohibitivo para los grafos de múltiples M-aristas que el motor ingiere regularmente. GraphHunter usa una variante con sampling: escoger un subconjunto uniforme aleatorio de S vértices de partida, correr BFS de camino más corto de fuente única desde cada uno y aproximar el betweenness como promedio de la contribución sobre la muestra.

Algorithm 13.1: COMPUTE-BETWEENNESS

Entrada: grafo G , tamaño de muestra S
Salida: mapa $B : V \rightarrow \mathbb{R}_{\geq 0}$

```
1 si  $|V| \leq S$  entonces
2   | sources  $\leftarrow V$ 
3 fin
4 sino
5   | sources  $\leftarrow$  muestra uniforme de  $S$  vértices
6 fin
7  $B \leftarrow \{v : 0 \ \forall v \in V\};$ 
8 para cada source  $s \in$  sources hacer
9   | BRANDES-SINGLE-SOURCE( $G, s, B$ );
10 fin
11 escalar cada entrada de  $B$  por  $|V|/|\text{sources}|$ ;
12 devolver  $B$ ;
```

Complejidad.

$O(V + E)$ en tiempo y espacio por source (BFS estándar + back-pass). El tiempo total con S sources es $O(S(V + E))$; para el default $S = 500$ y un grafo con $V = 10^5$ y $E = 10^6$, el costo es $\sim 5 \times 10^8$ operaciones — unos pocos cientos de ms en una laptop moderna. El espacio es $O(V + E)$: el estado BFS se reusa entre sources.

Algorithm 13.2: BRANDES-SINGLE-SOURCE

Entrada: grafo G , fuente s , acumulador B
 // Fase 1: BFS hacia adelante

```

1 para cada  $v$ :  $d[v] \leftarrow \infty$ ,  $\sigma[v] \leftarrow 0$ ,  $P[v] \leftarrow \emptyset$ ;
2  $d[s] \leftarrow 0$ ;  $\sigma[s] \leftarrow 1$ ; cola  $Q \leftarrow [s]$ ; pila  $St \leftarrow []$ ;
3 mientras  $Q$  no vacía hacer
4    $v \leftarrow Q.\text{pop\_front}$ ; push  $v$  en  $St$ ;
5   para cada vecino saliente  $u$  de  $v$  (vista no dirigida: salientes  $\cup$  entrantes) hacer
6     si  $d[u] = \infty$  entonces
7        $d[u] \leftarrow d[v] + 1$ ; push  $u$  en  $Q$ 
8     fin
9     si  $d[u] = d[v] + 1$  entonces
10       $\sigma[u] \leftarrow \sigma[u] + \sigma[v]$ ; anexar  $v$  a  $P[u]$ ;
11    fin
12  fin
13 fin
  
```

// Fase 2: back-accumulation

```

14 para cada  $v$ :  $\delta[v] \leftarrow 0$ ;
15 mientras  $St$  no vacía hacer
16    $w \leftarrow St.\text{pop}$ ;
17   para cada  $v \in P[w]$  hacer
18      $\delta[v] \leftarrow \delta[v] + \frac{\sigma[v]}{\sigma[w]}(1 + \delta[w])$ ;
19   fin
20   si  $w \neq s$  entonces
21      $B[w] \leftarrow B[w] + \delta[w]$ ;
22   fin
23 fin
  
```

La calidad esperada de la aproximación está acotada por Chernoff sobre una muestra aleatoria: con $S = O(\log V/\epsilon^2)$ sources, el betweenness por vértice se estima dentro de un factor multiplicativo $1 \pm \epsilon$ con alta probabilidad, asumiendo que la estructura de caminos más cortos no está patológicamente sesgada. El motor no intenta certificar esta cota online; el tamaño de muestra es una perilla configurable.

13.2. PageRank temporal

El PageRank temporal extiende al PageRank clásico con un decaimiento exponencial sobre los pesos de arista: una arista con timestamp $\tau(e)$ contribuye al rank computado al tiempo de referencia τ_0 con peso $\exp(-\lambda(\tau_0 - \tau(e)))$ si $\tau(e) \leq \tau_0$, cero en caso contrario.

Algorithm 13.3: COMPUTE-TEMPORAL-PAGERANK

Entrada: grafo G , decay λ , damping d , máx. iteraciones $I_{\text{máx}}$, tolerancia ϵ , tiempo de referencia τ_0

Salida: mapa $r : V \rightarrow [0, 1]$ que suma 1

```

1  $n \leftarrow |V|$ ; inicializar  $r[v] \leftarrow 1/n$  para todo  $v \in V$ ;
2 computar pesos de arista  $w(e) \leftarrow \exp(-\lambda \max(0, \tau_0 - \tau(e)))$ ;
3 computar pesos salientes normalizados  $w_{\text{out}}(v) \leftarrow \sum_{e:v \rightarrow \cdot} w(e)$ ;
4 para  $iter \leftarrow 1$  a  $I_{\text{máx}}$  hacer
5    $r' \leftarrow \{v : (1-d)/n \forall v\}$ ;
6   para cada vértice  $v$  con  $w_{\text{out}}(v) > 0$  hacer
7     para cada arista saliente  $e = (v \rightarrow u)$  hacer
8        $r'[u] \leftarrow r'[u] + d \cdot r[v] \cdot \frac{w(e)}{w_{\text{out}}(v)}$ ;
9     fin
10  fin
11  distribuir masa de nodos colgantes (vértices con  $w_{\text{out}} = 0$ ) uniformemente sobre  $V$ ;
12  si  $\|r' - r\|_1 < \epsilon$  entonces
13    devolver  $r'$ ;
14  fin
15   $r \leftarrow r'$ ;
16 fin
17 devolver  $r$ ;
```

Complejidad.

Cada iteración es $O(V + E)$. La iteración de potencia sobre una cadena amortiguada converge geoméricamente con ratio d , así que la cantidad de iteraciones es $O(\log(1/\epsilon)/\log(1/d))$. Con $d = 0.85$ y $\epsilon = 10^{-6}$ el motor observa convergencia en 10–20 iteraciones en la práctica, dando tiempo total $O(20(V + E))$ — lineal en el tamaño del grafo con constante chica.

13.3. Schema de relaciones

`get\_relation\_schema` barre cada arista y agrupa en triples únicos $(\ell_V(\text{src}), \ell_E(e), \ell_V(\text{dst}))$, devolviendo un vector ordenado lexicográficamente. Lo usa el chequeo de compatibilidad del Capítulo 25.

Complejidad.

$O(E + T \log T)$ donde $T = |M|$ es la cantidad de triples únicos. T es diminuto en la práctica (cientos bajos) así que el costo del sort es nominal; el costo dominante es el barrido lineal de aristas.

Algorithm 13.4: GET-RELATION-SCHEMA

Entrada: grafo G **Salida:** vector ordenado de (src_type, rel_type, dst_type, count)

```

1  $M \leftarrow$  HashMap vacío;
2 para cada arista  $e = (u, v, k)$  en  $G$  hacer
3    $M[(\ell_V(u), \ell_E(e), \ell_V(v))] += 1$ ;
4 fin
5  $\text{out} \leftarrow M.\text{into\_iter}$  como vector;
6 ordenar out lexicográficamente;
7 devolver out;
```

En el código.

```

graph\_hunter\_core/src/analytics.rs:641 — compute\_betweenness.
graph\_hunter\_core/src/analytics.rs:746 — compute\_temporal\_pagerank.
graph\_hunter\_core/src/analytics.rs:1289 — get\_relation\_schema.
```

Capítulo 14

Deduplicación y paginación de caminos

El matcher puede devolver muchos caminos para una hipótesis laxa. Antes de que el analista los vea, se deduplican (según el modo pedido), se puntúan, se filtran, se ordenan y se paginan. `score\_and\_paginate\_paths` es la única función que hace todo eso; vive en `graph\_hunter\_core/src/analyti`

14.1. Modos de dedup

Definición 14.1 (Modos de dedup). `DedupMode` es uno de:

- **None** — se mantiene cada camino del matcher.
- **ByPath** — los caminos con secuencias de vértices idénticas se fusionan en un único representante (el de mayor score).
- **ByEndpoints** — los caminos que comparten el mismo par (v_0, v_ℓ) se fusionan, sin importar los vértices intermedios.

ByPath es el default para hunts renderizados en la UI: cuando el grafo subyacente tiene muchas aristas paralelas entre el mismo par de vértices, el matcher produce múltiples caminos idénticos a la vista (uno por arista) y el analista sólo quiere ver la topología una vez.

ByEndpoints se usa durante análisis en bulk: “cuántos pares distintos start-end de alcanzabilidad satisfacen esta hipótesis” — la granularidad correcta para contar escenarios de ataque únicos.

14.2. Algoritmo

Complejidad.

Sea $P = |\Pi|$ (conteo crudo) y $U = |\text{únicos}|$ tras dedup.

- Dedup en **ByPath/ByEndpoints** es $O(P \cdot \ell)$ (hashing la clave de largo ℓ); **None** es $O(P)$.
- Scoring es $O(U \cdot \ell)$ usando SCORE-PATH.
- Sort es $O(U \log U)$ comparaciones.
- Slicing es $O(s)$.

Total: $O(P \cdot \ell + U \log U + s)$. Espacio $O(U + s)$ por encima del input.

Algorithm 14.1: SCORE-AND-PAGINATE

Entrada: caminos Π , índice de página p , tamaño de página s , min_score opcional, modo M , scorer S

Salida: página de caminos + conteo total (pre-página)

// Paso 1: dedup según el modo

```

1 segun  $M$  hacer
2   caso None hacer
3      $\text{únicos} \leftarrow \Pi$ 
4   fin
5   caso ByPath hacer
6     clavar cada  $\pi \in \Pi$  por su tupla de vértices; quedarse con el  $\pi$  de mayor score por
       clave
7   fin
8   caso ByEndpoints hacer
9     clavar cada  $\pi \in \Pi$  por  $(v_0, v_\ell)$ ; quedarse con el  $\pi$  de mayor score por clave
10  fin
11 fin
    // Paso 2: puntuar
12 para cada  $\pi \in \text{únicos}$  hacer
13    $(\text{composite}, \text{breakdown}) \leftarrow S.\text{score\_path}(\pi)$ 
14 fin
    // Paso 3: filtro opcional
15 si  $\text{min\_score}$  es Some entonces
16   descartar  $\pi$  con  $\text{composite} < \text{min\_score}$ 
17 fin
    // Paso 4: sort DESC por (composite, max_node_score, total_score)
18 ordenar únicos;
    // Paso 5: slice
19  $\text{total} \leftarrow |\text{únicos}|$ ;
20 devolver  $(\text{únicos}[p \cdot s .. (p + 1) \cdot s], \text{total})$ ;

```

14.3. Estabilidad del sort

La clave de sort es una tupla

$$(\text{composite}(\pi), \max_{v \in \pi} \text{composite}(\{v\}), \text{total_score}(\pi))$$

ordenada en sentido descendente. La clave de tres vías importa porque los puntajes compuestos empatan seguido en caminos de longitud corta — las claves secundaria y terciaria desempatan determinísticamente. En particular el max-node score promueve caminos que contienen un único vértice fuertemente anómalo por sobre caminos uniformemente mediocres.

14.4. Contrato de paginación

Invariante (Estabilidad de página). Dos llamadas consecutivas a `get\_hunt\_page` con los mismos parámetros devuelven el mismo orden, suponiendo que el grafo subyacente no fue mutado. El orden de caminos es determinista porque todo desempate es una cantidad totalmente ordenada derivada del estado del grafo.

El arnés de paridad (Capítulo 29) fija este invariante con una fixture que corre un hunt dos veces y asevera igualdad elemento a elemento de la página retornada.

En el código.

`graph\_hunter\_core/src/analytics.rs:1096` — `score\_and\_paginate\_paths`.
`graph\_hunter\_core/src/analytics.rs` — variantes de `DedupMode` y helpers de derivación de clave.

Capítulo 15

Diff de hunts

`diff\_hunts` responde a la pregunta “¿qué cambió entre dos puntos en el tiempo en el conjunto de caminos que satisfacen esta hipótesis?” Es la base del panel “qué hay nuevo desde mi último chequeo” en la UI y de la automatización programada que alerta sobre cadenas de ataque recién aparecidas.

15.1. Variante clásica por tiempo de evento

El algoritmo base corre la hipótesis dos veces contra el grafo, acotando la ventana temporal a distintos upper-bounds, y diffea los conjuntos de caminos.

Algorithm 15.1: DIFF-HUNTS

Entrada: grafo G , hipótesis H , timestamp baseline t_{base} , timestamp actual t_{cur} con $t_{\text{base}} < t_{\text{cur}}$

Salida: (agregados: $\text{List}[\pi]$, quitados: $\text{List}[\pi]$)

- 1 $\Pi_{\text{base}} \leftarrow \text{SEARCH-TEMPORAL-PATTERN}(G, H, (0, t_{\text{base}}), \infty)$;
 - 2 $\Pi_{\text{cur}} \leftarrow \text{SEARCH-TEMPORAL-PATTERN}(G, H, (0, t_{\text{cur}}), \infty)$;
 - 3 $U_{\text{base}} \leftarrow \text{SCORE-AND-PAGINATE}(\Pi_{\text{base}}, 0, \infty, \text{None}, \text{ByPath}).\text{únicos}$;
 - 4 $U_{\text{cur}} \leftarrow \text{idem para } \Pi_{\text{cur}}$;
 - 5 $K_{\text{base}} \leftarrow \text{conjunto de claves de camino en } U_{\text{base}}$;
 - 6 $K_{\text{cur}} \leftarrow \text{idem para } U_{\text{cur}}$;
 - 7 $\text{agregados} \leftarrow U_{\text{cur}} \text{ filtrados por clave } \notin K_{\text{base}}$;
 - 8 $\text{quitados} \leftarrow U_{\text{base}} \text{ filtrados por clave } \notin K_{\text{cur}}$;
 - 9 **devolver** (agregados, quitados);
-

Complejidad.

Dos ejecuciones de hunt más una pasada de diferencia de conjuntos. Si T_{match} es el costo del matcher y P es la cantidad de caminos devueltos por cualquiera de los hunts, el total es $2T_{\text{match}} + O(P)$. Espacio $O(P)$ por encima del scratch propio del matcher.

15.2. Trampa semántica: datos que llegan tarde

La variante clásica diffea por *tiempo de evento*, no por *tiempo de conocimiento*: una arista cuyo timestamp de evento es del pasado pero que fue ingerida después de t_{base} aparecerá tanto en Π_{base} como en Π_{cur} , porque el upper-bound de la ventana temporal no filtra basado en cuándo el motor se enteró de la arista. Esta es la motivación original del trabajo MVCC P3 (Capítulo 19): un analista que cerró un ticket en t_{base} igual puede querer saber que una ingesta post-mortem reveló aristas nuevas dentro de la ventana que ya consideraba “cerrada”.

15.3. La variante por snapshot (diferida)

Una vez que aterrice la extensión de matcher P3 (SEARCH-TEMPORAL-PATTERN-AT), `diff\_by\_snapshot` se vuelve la alternativa más rica:

Algorithm 15.2: DIFF-BY-SNAPSHOT (diferida)

Entrada: grafo G , hipótesis H , ventana de evento (t_0, t_1) , snapshot baseline ι_{base} ,
snapshot actual ι_{cur}

Salida: listas de caminos (agregados, quitados)

- 1 $\Pi_{\text{base}} \leftarrow \text{SEARCH-TEMPORAL-PATTERN-AT}(G, H, (t_0, t_1), \iota_{\text{base}});$
 - 2 $\Pi_{\text{cur}} \leftarrow \text{SEARCH-TEMPORAL-PATTERN-AT}(G, H, (t_0, t_1), \iota_{\text{cur}});$
 - 3 ... set-diff como en DIFF-HUNTS ...
-

Distinción clave: la ventana de evento (t_0, t_1) es fija, y lo que varía es el snapshot de conocimiento ι . El motor responde “¿qué caminos nuevos en esta ventana de evento me enteré entre ι_{base} y ι_{cur} ?” — la semántica apropiada de auditoría para enriquecimiento que llega tarde, detonaciones VirusTotal que vuelven después de ingerido el evento, overlays de threat intel que llegaron más tarde, etc.

Complejidad.

Igual que la variante clásica: $2T_{\text{match}} + O(P)$. El delta de *valor* es semántico, no de performance.

15.4. Validación de entrada

Invariante (Orden de la ventana). `diff\_hunts` rechaza $t_{\text{base}} > t_{\text{cur}}$ con `ApiError::InvalidInput`. Una ventana invertida produciría un par (agregados, quitados) cuya semántica es responsabilidad del usuario de interpretar; el motor prefiere fallar ruidosamente.

Este invariante lo ejerce el arnés de paridad (`diff\_hunts\_rejects\_inverted\_window` en `graph\_hunter\_api/tests/parity/scenarios.rs`).

En el código.

`graph\_hunter\_core/src/analytics.rs:1342` — `diff\_hunts` (clásico).
`graph\_hunter\_core/src/relation.rs:83` — el campo `ingested\_at\_delta` que `diff\_by\_snapshot` eventualmente consultará.

Parte IV

Pipeline de ingesta

Capítulo 16

El trait LogSource

Nota. Este capítulo es un *stub*. El trait `LogSource` es la capa inferior del pipeline de ingesta streaming: cada implementación representa un productor externo de datos (un workspace Microsoft Sentinel, un tenant Defender XDR, una fuente blob Azure Data Lake). Un recorrido completo del ciclo de vida del polling — semántica de retomar en `poll\since`, cancelación, monotonía de watermark, backoff y quota — queda diferido a una revisión posterior.

16.1. Estructura

```
#[async_trait]
pub trait LogSource: Send + Sync {
    fn source_id(&self) -> &str;
    fn kind(&self) -> SourceKind;
    fn available_tables(&self) -> &[&'static str];
    async fn check_auth(&self) -> Result<(), SourceError>;
    async fn query_scoped(
        &self,
        scope: QueryScope,
        cancel: CancellationToken,
    ) -> Result<ChunkStream, SourceError>;
    async fn poll_since(
        &self,
        table: &str,
        watermark: Watermark,
        cancel: CancellationToken,
    ) -> Result<PollStream, SourceError>;
}
```

Un `RawChunk` es un fragmento de respuesta respaldado por `Bytes` que lleva `source\id`, `table`, `fetches\at`, y un `watermark\candidate` opcional que la fuente sugiere persistir tras un parseo exitoso.

16.2. Implementaciones

- `graph\hunter\core/src/sources/log\_analytics.rs` — Microsoft Sentinel / Azure Log Analytics.
- `graph\hunter\core/src/sources/defender\_xdr.rs` — Microsoft Defender XDR (Advanced Hunting).
- `graph\hunter\core/src/sources/data\_lake.rs` — enumeración de blobs en Azure Data Lake.

- `graph\_hunter\_core/src/sources/aad\_training\_export.rs` — pipeline de exportación de entrenamiento Azure AD.

En el código.

`graph\_hunter\_core/src/sources/mod.rs:275` — definición del trait.
`graph\_hunter\_core/src/sources/token\_cache.rs` — cacheo de tokens Azure.
`graph\_hunter\_core/src/sources/backoff.rs` — política de backoff exponencial.

Capítulo 17

Parsers

Nota. Stub. El trait `LogParser` y sus implementaciones por formato (Sentinel JSON, Sysmon XML, Cognito, IIS W3C, FortiAnalyzer XML, CSV/JSON genérico con mapeos de campo) merecen cada uno un capítulo; esta sección lista el seam y las implementaciones conocidas.

17.1. Trait

```
pub trait LogParser: Send + Sync {
    fn parse(&self, data: &str) -> Vec<ParsedTriple>;
    fn parse_rows(&self, rows: &[serde_json::Value]) -> Vec<ParsedTriple>
    {
        // Default: serializar rows de vuelta a JSON array y delegar.
    }
}

pub type ParsedTriple = (Entity, Relation, Entity);
```

El override `parse\_rows` existe para cortocircuitar el round-trip “JSON → string → JSON” en los parsers que ya consumen el shape `serde\_json::Value` (notablemente `SentinelJsonParser`).

17.2. Implementaciones

- `graph\_hunter\_core/src/sentinel.rs` — `SentinelJsonParser` (también sobrescribe `parse\_rows`).
- `graph\_hunter\_core/src/sysmon.rs` — parseo de eventos XML de Sysmon.
- `graph\_hunter\_core/src/cognito.rs` — parseo de auditoría AWS Cognito.
- `graph\_hunter\_core/src/iis\_w3c.rs` — formato de log IIS W3C.
- `graph\_hunter\_core/src/forti\_analyzer.rs` — XML FortiAnalyzer con registro de handlers por sección.
- `graph\_hunter\_core/src/generic.rs` — CSV/JSON genérico con un `FieldMapping` que describe qué columna cumple qué rol semántico.
- `graph\_hunter\_core/src/csv\_parser.rs` — helper CSV liviano usado por `GenericParser`.

17.3. Despacho de parser

El motor no hace autodetección de formato en el path del matcher; en cambio, la fachada de ingesta (`graph\_hunter\_api`) elige un parser por `RawChunk.encoding` (Json / Jsonl /

Parquet) más registro por fuente. La familia `preview` (`graph\_hunter\_core/src/preview.rs`, `field\_preview.rs`) permite a la UI olfatear un archivo y sugerir un mapeo antes de comprometerse con la ingesta.

En el código.

```
graph\_hunter\_core/src/parser.rs — trait LogParser.
```

Capítulo 18

Writer y backpressure

Nota. Stub. El task writer es el único punto de mutación para el `InMemoryGraph` durante ingesta streaming. Un tratamiento completo (preempción por enriquecimiento, contabilidad de cuota, overflow a disco) se difiere.

18.1. Forma del pipeline

```
LogSource -> raw_channel -> ParserTask -> parsed_channel -> GraphWriter -> Graph
```

Ambos canales son colas `tokio::sync::mpsc` acotadas (capacidades default 4 y 8 respectivamente, configurables por fuente). Un canal lleno hace backpressure sobre el stage aguas arriba; una fuente cuyo poll no encuentra lugar en `raw_channel` se bloquea en `send().await` hasta que el writer haya drenado lo suficiente como para avanzar.

18.2. Módulos

- `graph\_hunter\_core/src/ingest/batch.rs` — unidad de trabajo `ParsedBatch` (triples + source_id + dataset_id + prioridad + watermark).
- `graph\_hunter\_core/src/ingest/parser\_task.rs` — `ParserTask` consume `raw\_channel`, llama al `LogParser` específico del formato y emite `ParsedBatch`.
- `graph\_hunter\_core/src/ingest/writer.rs` — `GraphWriter`: task singleton que drena `parsed\_channel`, toma el write lock del grafo por un batch a la vez, y lo libera entre batches para que los lectores puedan progresar.
- `graph\_hunter\_core/src/ingest/source\_poller.rs` — `SourcePoller`: loop con timer por fuente que maneja `LogSource::poll\_since`.
- `graph\_hunter\_core/src/ingest/enrichment.rs` — batches de enriquecimiento on-demand, preemptados por encima de la ingesta bulk.
- `graph\_hunter\_core/src/ingest/channels.rs` — constructores de canales acotados.
- `graph\_hunter\_core/src/ingest/overflow.rs` — serializa `ParsedBatches` pendientes a disco en cancelación; replay en reinicio (Capítulo 20).

18.3. Contención de locks

El writer toma `graph.write()` durante una sola llamada a `insert\_triples\_chunk` y la libera entre batches. Los lectores (hunts, queries analíticas, queries de enriquecimiento) toman

`graph.read()` y coexisten entre sí. Los tamaños de batch típicos son 1–5 K triples, así que un hold de write dura milisegundos — los lectores retroceden una vez por batch, lo cual es barato.

Invariante (Unicidad del writer). Cada sesión tiene a lo sumo un task `GraphWriter` activo en un momento dado. El `SessionStore` registra el `JoinHandle` del writer cuando arranca el task y rechaza arrancar un segundo; esto descarta totalmente races writer-writer.

En el código.

`graph\_hunter\_core/src/ingest/writer.rs` — loop principal del writer.

Capítulo 19

Estampado MVCC a nivel arista

Nota. Stub. El refactor P3 introdujo MVCC a nivel arista con overhead de memoria cero, recuperando padding en `CompactRelation` (ver Capítulo 3 para el layout y Capítulo 15 para la semántica de diff con snapshots). Este capítulo eventualmente describirá el comportamiento de estampado del pipeline de ingesta, la extensión de matcher `search\_temporal\_pattern\_at`, y la migración de archivo de sesión / archivo de spill `v1→v2`.

19.1. Estado actual

- `CompactRelation.ingested\_at\_delta`: `u32` lo estampa `GraphHunter::add\_relation` con el segundo wall-clock actual al momento de insertar. Las aristas heredadas (`delta = 0`) resuelven al `INGEST\_EPOCH\_BASE`.
- `encode\_ingested\_at(ts)` es el helper del path de ingesta que clampa por debajo del epoch base y satura por encima de `u32::MAX`.
- El invariante de 32 bytes sobre `CompactRelation` es verificado por CI (`relation::tests::compact\_relat`).

19.2. Trabajo pendiente

- `SEARCH-TEMPORAL-PATTERN-AT( $G, H, (t_0, t_1), \iota$ )` — el wrapper de matcher que filtra aristas por `ingested\_at() ≤  $\iota$` además de la ventana de evento. Bump de costo esperado: 5–15 % en queries con snapshot; transparente sobre queries de vista actual.
- Propagación `IngestAdapter → writer`: pasar `TripleBatch.fetched\_at` a `with\_ingested\_at` así las fuentes streaming reciben el timestamp upstream en vez del wall-clock local. Determinista para replay.
- Función analítica `diff\_by\_snapshot` (Capítulo 15).
- Bump de `schema\_version` del `SessionFile` (`v1→v2`): backfill de `ingested\_at` desde `timestamp` para aristas heredadas, mostrar banner UI “snapshot aproximado”.
- Bump de `SPILL\_VERSION` y migrador CLI.

En el código.

`graph\_hunter\_core/src/relation.rs:83` — layout de `CompactRelation` con campo MVCC.
`graph\_hunter\_core/src/graph.rs:227` — estampado de `add\_relation` hoy.
`graph\_hunter\_core/tests/mvcc\_ingested\_at.rs` — fijación del contrato MVCC end-to-end.

Capítulo 20

Persistencia de desborde

Nota. Stub. Cuando el pipeline de ingesta se cancela (el usuario cierra la sesión, el proceso termina) o cuando el `parsed\_channel` se llena más rápido de lo que el writer puede drenar, los `ParsedBatches` encolados se persisten a disco en vez de descartarse. En la próxima carga de sesión el desborde se reproduce en orden antes de que el streaming se reanude.

20.1. Invariantes

- Cada batch persistido retiene su `source\_id`, `watermark` y prioridad, así que la reanudación avanza el watermark de la fuente exactamente como lo haría una ingesta en caliente.
- El replay es idempotente: el grafo deduplica por $(source, dest, rel, timestamp)$ en `GraphWriter::insert\_` así que el mismo batch aplicado dos veces tiene el mismo efecto que una sola.

20.2. Formato de archivo

Los batches de desborde se guardan bajo el directorio de sesión como archivos JSON. El formato es estable entre reinicios y deliberadamente legible para humanos: un operador puede inspeccionar el trabajo encolado con `jq` sin más herramientas que un editor de texto.

En el código.

`graph\_hunter\_core/src/ingest/overflow.rs` — persistencia y replay.

Parte V

Puntos de extensión

Capítulo 21

Traits GraphRead / GraphWrite

Nota. Stub. El refactor P2-B extrajo estos traits para que los consumidores del motor (scorer de anomalía, extractor de features GNN, writer de ingesta) puedan depender de una API semántica en vez del struct concreto `GraphHunter`. El objetivo eventual es poder enchufar backends remotos (Neo4j, un store MVCC streaming) sin editar el scorer, el matcher ni el writer — con tal de que existan las impls del trait.

21.1. Traits

```
pub trait GraphRead {
    fn entity_count(&self) -> usize;
    fn relation_count(&self) -> usize;
    fn get_entity(&self, id: &str) -> Option<&Entity>;
    fn get_compact_relations(&self, src: &str)
        -> Cow<'_, [CompactRelation]>;
    fn get_reverse_source_ids(&self, id: &str) -> Vec<String>;
    fn entity_ids_for_type(&self, et: &EntityType) -> Option<Vec<String>>;
    fn entity_types_in_graph(&self) -> Vec<String>;
    fn materialize_relation(&self, c: &CompactRelation) -> Relation;
    fn supports_simd(&self) -> bool { false }
}

pub trait GraphWrite: GraphRead {
    fn add_entity(&mut self, e: Entity) -> Result<(), GraphError>;
    fn add_relation(&mut self, r: Relation) -> Result<(), GraphError>;
}
```

El retorno `Cow<'_, [CompactRelation]>` es la clave que permite coexistencia de backends remotos: `InMemoryGraph` devuelve `Cow::Borrowed(&slice)` con costo cero; una impl remota devuelve `Cow::Owned(vec)` cuando tiene que sintetizar la lista.

21.2. Qué va en el trait y qué no

Va: lectura y escritura semánticas, las operaciones que un hipotético backend remoto necesitaría reimplementar.

No va: el matcher en sí (`search\_temporal\_pattern`), el despacho SIMD, los analíticos (`pagerank`, `betweenness`). Son específicos del motor; un backend Neo4j traería su propio motor de patrones en vez de implementar el DFS de Rust contra un `\&dyn GraphRead`.

21.3. Contrato dual-impl

`graph\_hunter\_core/tests/backend\_dual\_impl.rs` corre un arnés genérico `validate\_backend<B: GraphWrite>(b)` que ejercita cada método del trait contra `GraphHunter`. Cuando aterrice una segunda impl, el mismo arnés la cubre gratis.

En el código.

`graph\_hunter\_core/src/backend/mod.rs` — definiciones del trait.
`graph\_hunter\_core/src/backend/in\_memory.rs` — `impl GraphRead + GraphWrite for GraphHunter`.
`graph\_hunter\_core/tests/backend\_dual\_impl.rs` — validación dual-impl.

Capítulo 22

Traits Scorer y ScoreComponent

Nota. Stub. El refactor P2-A extrajo cada dimensión de scoring en una impl `ScoreComponent` separada. Ver Capítulo 12 para la matemática subyacente.

22.1. Traits

```
pub trait ScoreComponent: Send + Sync {
    fn name(&self) -> &'static str;
    fn score_path(&self, ctx: &PathContext<'_>) -> f64;
    fn explain(&self, value: f64, ctx: &PathContext<'_>) -> Option<String>
        {
            None
        }
}

pub struct ScoringCaches<'a> {
    pub entity_freq: &'a HashMap<String, u64>,
    pub pair_freq: &'a HashMap<(String, String), u64>,
    pub first_seen: &'a HashMap<String, i64>,
    pub nc_cache: &'a HashMap<String, f64>,
    pub gnn_cache: &'a HashMap<String, f64>,
    pub log_max_freq: f64,
    pub log_max_pair_freq: f64,
    pub tau_min: i64,
    pub tau_max: i64,
}

pub struct PathContext<'a> {
    pub path: &'a [String],
    pub caches: &'a ScoringCaches<'a>,
}
```

Cinco componentes built-in: `EntityRarity`, `EdgeRarity`, `NeighborhoodConcentration`, `TemporalNovelty`, `GnnThreat`. Cada uno vive en `graph\_hunter\_core/src/scoring/components.rs` como struct unitario que implementa `ScoreComponent`.

22.2. Orquestador compuesto

`CompositeScorer` sostiene `Vec<Box<dyn ScoreComponent>` más pesos coincidentes y hace un promedio ponderado $\sum w_i c_i / \sum w_i$. Peso cero desactiva un componente (igual se evalúa pero contribuye 0).

22.3. Migración

`AnomalyScorer::score\_path\_v2` es una alternativa drop-in al `score\_path` hardcodeado. Ambos coexisten por un release; `graph\_hunter\_core/tests/scoring\_snapshot.rs` fija equivalencia bit-a-bit. Un follow-up flipa el default y elimina v1.

En el código.

```
graph\_hunter\_core/src/scoring/mod.rs — trait + ScoreOutcome.  
graph\_hunter\_core/src/scoring/composite.rs — CompositeScorer.  
graph\_hunter\_core/src/scoring/components.rs — 5 impls de componentes.
```

Capítulo 23

Trait StringInternerBackend

Nota. Stub. Ver Capítulo 4 para el struct y las métricas subyacentes. El trait existe como seam para un futuro `GenerationalInterner` o `PartitionedInterner`; no se entrega una segunda impl hoy, el trait está pre-posicionado para cuando la evidencia de producción lo justifique.

23.1. Contrato

```
pub trait StringInternerBackend {
    fn intern(&mut self, s: &str) -> StrId;
    fn resolve(&self, id: StrId) -> &str;
    fn try_resolve(&self, id: StrId) -> Option<&str>;
    fn get(&self, s: &str) -> Option<StrId>;
    fn len(&self) -> usize;
    fn is_empty(&self) -> bool;
    fn reserve(&mut self, additional: usize);
    fn metrics(&self) -> InternerMetrics;
}

pub struct InternerMetrics {
    pub unique_strings: usize,
    pub approx_bytes:  usize,
}
```

23.2. Condiciones disparadoras

Implementar un segundo backend vale la pena cuando cualquiera de:

- `metrics().unique\_strings` excede $\sim 10^7$ en una sesión viva.
- Una sesión streaming corre 24×7 por más de 72h sin compactación.
- Un único proceso sirve a múltiples tenants (despliegue MSSP).

El motor expone las métricas vía `api.get\_interner\_metrics()` para que un operador pueda observar la aproximación a un gatillo en vez de chocarse con él a ciegas.

En el código.

`graph\_hunter\_core/src/interner.rs` — trait + métricas.
`TODO.md` — criterios de disparo.

Capítulo 24

Trait IngestAdapter

Nota. Stub. El refactor P2-E introdujo un wrapper de más alto nivel sobre `LogSource + LogParser`: los autores de una fuente nueva implementan `IngestAdapter` en una crate hija en lugar de editar archivos del core.

24.1. Contrato

```
#[async_trait]
pub trait IngestAdapter: Send + Sync {
    fn adapter_id(&self) -> &str;
    fn tables(&self) -> &[&'static str];
    async fn stream_triples(
        &self,
        table: &str,
        since: Watermark,
        cancel: CancellationToken,
    ) -> Result<TripleBatchStream, SourceError>;
}

pub struct TripleBatch {
    pub triples: Vec<ParsedTriple>,
    pub watermark: Watermark,
    pub fetched_at: i64,
}
```

24.2. Implementación genérica

`LogSourceAdapter` envuelve cualquier `LogSource + LogParser` en un `IngestAdapter`. Las 4 fuentes existentes (`Log Analytics`, `Defender XDR`, `Data Lake`, `AAD export`) pueden exponerse como adapters sin cambio de código fuente; el orquestador aún las sostiene de forma concreta hoy por compatibilidad hacia atrás, pero el seam está abierto.

24.3. Propagación MVCC

`TripleBatch.fetched_at` lleva el timestamp de fetch upstream, que es exactamente lo que el writer necesita para llamar a `CompactRelation::with_ingested_at` en cada arista insertada — la tercera pata de la historia MVCC P3 (Capítulo 19). Cablear esa plomería end-to-end queda como follow-up diferido.

En el código.

```
graph\_hunter\_core/src/ingest\_adapter.rs — trait + LogSourceAdapter.
```

Capítulo 25

Cargador de catálogo YAML

Nota. Stub. El refactor P2-C movió el catálogo de hipótesis ATT&CK de un array `const` en compile time a 29 YAMLs embebidos más un directorio opcional de override de usuario.

25.1. Precedencia

Las fuentes posteriores sobrescriben a las anteriores por id de catálogo:

1. **Built-in** — 29 archivos embebidos vía `include\_str!` desde `graph\_hunter\_core/src/catalog/builtin/`
2. **Directorio de usuario** — `\$GRAPHHUNTER\_CATALOG\_DIR` si está seteado; si no `\$USERPROFILE/.graphh` (Windows) o `\$HOME/.graphhunter/catalog/` (Unix). Silencioso si no existe.

25.2. Validación

Cada entrada cargada pasa por una validación:

- `id` matchea `[A-Za-z0-9\-\_]+`.
- `mitre\_id` matchea `T\textbackslash d\{4\}(\textbackslash .\textbackslash d\{3\})?`.
- `dsl\_pattern` parsea vía `parse\_dsl`.
- `k\_simplicity` `>= 1`.

Los archivos de usuario malformados no hacen panic ni rompen el startup: el cargador loguea un `CatalogLoadError` y sigue. Los YAMLs built-in los cubre estructuralmente un golden test (`graph\_hunter\_core/tests/catalog\_golden.rs`) y el chequeo de compile time `include\_str!`.

25.3. Diagnósticos

`api.get\_catalog\_diagnostics()` devuelve el `Vec<CatalogLoadError>` recolectado. Los transports lo exponen vía `cmd\_get\_catalog\_diagnostics` (Tauri) y `GET /catalog/diagnostics` (HTTP).

En el código.

`graph\_hunter\_core/src/catalog/mod.rs` — API pública (`get\_catalog`, `get\_catalog\_snapshot`).
`graph\_hunter\_core/src/catalog/loader.rs` — carga + validación.
`graph\_hunter\_core/src/catalog/builtin/*.yaml` — 29 entradas built-in.

Parte VI

Capa de transporte

Capítulo 26

La fachada canónica de API

Nota. Stub. `GraphHunterApi` es el único objeto del que cada transport (Tauri, HTTP, MCP) mantiene un `Arc`. Toda la gestión de sesión, cache de hunt, conversación AI, streaming Sentinel, scorer GNN y estado GeoIP vive dentro de `ApiState`.

26.1. Estado

```
pub struct GraphHunterApi(Arc<ApiState>);

struct ApiState {
    sessions:           SessionStore,
    hunt_cache:         Arc<RwLock<Vec<Vec<String>>>>,
    npu_scorer:          Arc<RwLock<Option<NpuScorer>>>,
    geoip_provider:      Arc<Mutex<Option<MaxMindProvider>>>,
    http_geoip_client:   Arc<Mutex<Option<Arc<dyn HttpClient>>>>,
    sentinel_connector:  Arc<RwLock<Option<SentinelConnectorHandle>>>,
    ai_config:           Arc<RwLock<Option<ProviderConfig>>>,
    emitter:             Arc<dyn EventEmitter>,
    // ... 8 campos mas
}
```

La fachada la introdujo el refactor P1. Antes, cada transport sostenía su propio estado con forma distinta; con la fachada en su lugar, Tauri / HTTP / MCP son estrictamente capas de delegación.

26.2. Módulo de operaciones

Cada método de `GraphHunterApi` se define en `graph\_hunter\_api/src/operations/*.rs`, un archivo por categoría (analytics, anomaly, dataset, dsl, enrichment, entity, export, geoip, graph_ops, hunt, ingestion, sentinel, session, sigma, ticket, ai). Un método acepta un request DTO, despacha la sesión si hace falta, llama al motor y devuelve un response DTO.

26.3. Handle de sesión

`SessionHandle(String)` es un ID opaco de sesión; los requests o lo llevan (target explícito) o lo omiten (usar la sesión actual). La fachada resuelve el handle en un solo lugar (`SessionStore::resolve`) así toda operación hereda la misma lógica de precedencia.

En el código.

`graph\_hunter\_api/src/lib.rs` — fachada `GraphHunterApi`, `ApiState`.
`graph\_hunter\_api/src/state/session.rs` — `Session`, `SessionHandle`, `SessionStore`.

Capítulo 27

DTOs

Nota. Stub. Cada operación entre un transport y la fachada usa un par DTO (`*Request + *Response`) que vive en `graph\_hunter\_api/src/dto/`. Los DTOs son el contrato a nivel wire: si un campo se renombra acá, el output serde de cada transport cambia igual, y el arnés de paridad (Capítulo 29) atrapa el drift en tiempo de test.

27.1. Layout del módulo

Un archivo por categoría de operación, espejando `operations/`: `ai.rs`, `analytics.rs`, `anomaly.rs`, `dataset.rs`, `dsl.rs`, `enrichment.rs`, `entity.rs`, `export.rs`, `geoip.rs`, `graph\_ops.rs`, `hunt.rs`, `ingestion.rs`, `sentinel.rs`, `session.rs`, `sigma.rs`, `ticket.rs`.

27.2. Patrones comunes

- Cada request DTO que toca estado de sesión lleva `session: Option<SessionHandle>`. La fachada resuelve `None` a la sesión actual.
- Los errores fluyen vía `Result<T, ApiError>` (ver `graph\_hunter\_api/src/error.rs`), que cada transport mapea a su propio envelope (HTTP 400/404/500, `Tauri CommandError`, texto de error MCP).
- Las variantes de enum usan `\#[serde(tag = "kind")]` para variantes struct; las variantes newtype se serializan como su string Display plano (una limitación de serde que documentamos en el test de roundtrip de errores del Capítulo 29).

27.3. Invariantes de forma

La *Fase A* del arnés de paridad fija el conjunto de claves de cada DTO: un rename cambia silenciosamente el JSON enviado al wire, y dos transports contruidos sobre la misma fachada seguirían en desacuerdo si uno hubiera cacheado una copia vieja del DTO.

En el código.

`graph\_hunter\_api/src/dto/*.rs` — todos los DTOs.
`graph\_hunter\_api/src/error.rs` — `ApiError`.
`graph\_hunter\_api/tests/parity/dto\_shapes.rs` — fijación de forma Fase A.

Capítulo 28

Adaptadores de transporte

Nota. Stub. Tres transports delegan en `GraphHunterApi`: la app desktop Tauri, el servidor HTTP basado en axum, y el servidor MCP en TypeScript. Los tres están pensados para ser delgados — nada de lógica más allá de construir un request DTO y renderizar la respuesta.

28.1. Tauri

Los comandos viven en `app/src-tauri/src/commands/*.rs`. Cada cuerpo de comando es ≤ 10 LOC:

```
#[tauri::command]
pub fn cmd_run_hunt(
    api: State<Arc<GraphHunterApi>>,
    hypothesis: Hypothesis,
    time_window: Option<(i64, i64)>,
    max_results: Option<u32>,
) -> Result<RunHuntResponse, CommandError> {
    api.run_hunt(RunHuntRequest {
        session: None,
        hypothesis,
        time_window,
        max_results,
    })
    .map_err(CommandError::from)
}
```

Los comandos se registran en `app/src-tauri/src/lib.rs` con la macro `tauri::Builder::invoke_handler`.

28.2. HTTP

Los handlers viven en `app/src-tauri/src/http/_api.rs` y comparten el mismo `Arc<GraphHunterApi>` vía el trait `FromRef` de axum. El puerto HTTP se elige al arrancar (`util::resolve_api_port`) y se escribe al directorio de config de la app Tauri para que el servidor MCP y la CLI puedan encontrarlo.

Cada handler también es ≤ 10 LOC:

```
async fn handler_run_hunt(
    State(api): State<Arc<GraphHunterApi>>,
    Json(body): Json<RunHuntRequest>,
) -> Response {
    match api.run_hunt(body) {
        Ok(v) => ok_json(v),
        Err(e) => error_json(e),
    }
}
```

```
}
}
```

28.3. MCP

`graph-hunter-mcp/src/index.ts` expone ~30 herramientas sobre el protocolo MCP. Cada handler de tool hace una llamada HTTP al motor local vía `http://127.0.0.1:<port>`; no enlaza el motor Rust en absoluto. Este es el único transport que no sostiene un `Arc<GraphHunterApi>` directo — porque corre en otro proceso (Node.js) — pero ve exactamente la misma semántica porque la capa HTTP es un wrapper delgado de la fachada.

28.4. EventEmitter

Los eventos de progreso (carga de sesión, ingesta, streaming Sentinel) fluyen por un `Arc<dyn EventEmitter>` guardado en `ApiState`. `TauriEventEmitter` puentea a `AppHandle::emit`; `NoopEmitter` lo usan HTTP, MCP, CLI y tests.

En el código.

`app/src-tauri/src/commands/` — módulos de comandos Tauri.
`app/src-tauri/src/http\_api.rs` — handlers HTTP.
`graph-hunter-mcp/src/index.ts` — tools MCP.
`graph\_hunter\_api/src/events.rs` (o similar) — trait `EventEmitter`.

Capítulo 29

Arnés de paridad

Nota. Stub. El arnés vive en `graph\_hunter\_api/tests/parity*.rs`. Dos fases se entregan hoy (A: forma de DTO; B: escenario); una tercera cubriendo el runtime HTTP + Tauri queda diferida.

29.1. Fase A — forma de DTO

Para cada par de DTO request/response, el arnés serializa un valor armado a mano a JSON y asevera que las claves top-level del objeto coincidan exactamente con un conjunto esperado. Un campo que falta o un rename levanta un error claro en test time.

El helper:

```
fn assert_keys<T: Serialize>(value: &T, expected: &[&str]) {
    let v = serde_json::to_value(value).unwrap();
    let actual: BTreeSet<_> = v.as_object().unwrap()
        .keys().map(|s| s.as_str()).collect();
    let want: BTreeSet<_> = expected.iter().copied().collect();
    assert_eq!(actual, want);
}
```

29.2. Fase B — escenario

Los escenarios end-to-end ejercitan secuencias ordenadas de operaciones contra la fachada directamente (no a través de un transport). El arnés construye una sesión fixture con un mini-grafo seed (Alice + Bob autenticándose a web-01, web-01 ejecutando `cmd.exe`), corre una secuencia de llamadas de API y asevera estado observable en cada paso.

Fixtures:

- `fresh\_api\_with\_session()` — API fresca vacía más una sesión única.
- `fresh\_api\_with\_seeded\_session()` — el grafo seed de 4 entidades / 3 aristas.
- `tag`, `note`, `parse` — helpers de conveniencia para que los escenarios se lean al nivel de operación.

Aserciones de muestra: `run\_hunt\_populates\_cache\_and\_pages\_back` (un run completo → round-trip de page-back matchea), `diff\_hunts\_rejects\_inverted\_window` (el invariante del Capítulo 15), `export\_iocs\_sentinel\_watchlist\_csv\_contains\_tagged\_ips\_only` (el formato de export).

29.3. Fase C diferida

Un arnés runtime HTTP + Tauri completo (levantar axum en el test, cablear un mock de IPC Tauri) está marcado como follow-up. Fase A + B juntas atrapan los drifts de forma y semántica que habrían roto la paridad entre transports; Fase C agregaría round-trips a nivel de wire.

En el código.

`graph\_hunter\_api/tests/parity.rs` — entry del arnés.
`graph\_hunter\_api/tests/parity/fixture.rs` — constructores de fixtures.
`graph\_hunter\_api/tests/parity/dto\_shapes.rs` — Fase A.
`graph\_hunter\_api/tests/parity/scenarios.rs` — Fase B.

Capítulo 30

Persistencia de sesión

Nota. Stub. Las sesiones se serializan a archivos JSON bajo el directorio de datos de app del SO. El formato es hoy `schema_version = 1`; un bump v2 para la migración MVCC P3 está planeado (Capítulo 19).

30.1. Layout en disco

- Windows: `\\%APPDATA%\\textbackslash GraphHunter\\textbackslash sessions\\textbackslash <id>.json`
- Unix: `\\$HOME/.local/share/GraphHunter/sessions/<id>.json`

`<id>` se valida contra `[A-Za-z0-9\\_\\-]\\{1,128\\}` para prevenir path traversal. El path se construye vía `PathBuf::join`, nunca por concatenación de strings.

30.2. Forma del SessionFile

```
pub struct SessionFile {
    pub id: String,
    pub name: String,
    pub created_at: i64,
    pub entities: Vec<Entity>,
    pub relations: Vec<Relation>,
    pub path_node_ids: Vec<String>,
    pub notes: Vec<Note>,
    pub datasets: Vec<DatasetInfo>,
    pub tags: Vec<EntityTag>,
    // SIN ai_conversation -- el chat es deliberadamente efimero.
}
```

30.3. Progreso de carga

La carga es asíncrona. El `EventEmitter` recibe eventos `SESSION\\_LOAD\\_PROGRESS` en cada etapa: parseo del archivo, inserción de entidades, inserción de relaciones, reconstrucción de índices (finalize del scorer), listo. La UI renderiza una barra de progreso; HTTP / MCP simplemente ignoran los eventos (`NoopEmitter`).

30.4. Migración v2 (diferida)

Cuando aterrice el cambio de writer MVCC P3, las aristas heredadas (cargadas de archivos v1) deben backfillarse con un `ingested\_at` sintético. El plan es usar el `timestamp` de la arista como valor de backfill: no es exactamente correcto (la arista fue ingerida en algún momento posterior), pero es una elección definida y determinista que permite a las queries de snapshot v2 producir resultados sensatos contra datos v1. La UI muestra un banner “snapshot aproximado” para que el analista lo sepa.

En el código.

```
graph\_hunter\_api/src/state/persistence.rs — load / save.  
graph\_hunter\_api/src/state/session\_types.rs — SessionFile, Note, EntityTag.
```

Apéndice A

Resumen de complejidad

Referencia de una página sobre las operaciones dominantes del motor, con complejidad temporal y espacial. Las referencias de capítulo apuntan a la derivación completa.

Operación	Tiempo	Espacio	Cap.	Archivo
INTERN	$O(1)$ am.	$O(s)$ miss	4	interner.rs:60
RESOLVE	$O(1)$	—	4	interner.rs:74
ADD-ENTITY	$O(1)$ am.	$O(1)$	5	graph.rs:211
ADD-RELATION	$O(1)$ am.	$O(32)$	5	graph.rs:227
SORT-EDGES-BY-TS	$O(m \log \bar{d})$	$O(1)$	5	graph.rs:437
SPILL-APPEND	$O(1)$ am.	$O(L)$	6	spill.rs
K-WAY-MERGE	$O(n \log k)$	$O(k)$	6	spill.rs:327
METADATA-APPEND	$O(K)$	$O(K)$	7	metadata_store.rs
PARSE-DSL	$O(src)$	$O(src)$	9	dsl.rs:47
DFS-MATCH-ITERATIVE	$O(n_s \bar{b}^\ell \ell)$	$O(W \ell k)$	10	graph.rs:1121
MERGE-INTO-TOPK	$O((a + b) \log K)$	$O(K)$	10	graph.rs:782
INTERSECT-SORTED-U32	$O(S L /8)$	$O(1)$	11	simd_rust.rs
ANOMALY-FINALIZE	$O(V + E)$	$O(V)$	12	anomaly.rs:148
COMPUTE-NC	$O(d(v))$	$O(\mathcal{T}_V)$	12	anomaly.rs:191
SCORE-PATH	$O(\ell)$	$O(1)$	12	anomaly.rs:241
COMPUTE-BETWEENNESS	$O(S(V + E))$	$O(V + E)$	13	analytics.rs:641
TEMPORAL-PAGERANK	$O(I(V + E))$	$O(V)$	13	analytics.rs:746
GET-RELATION-SCHEMA	$O(E + T \log T)$	$O(T)$	13	analytics.rs:1289
SCORE-AND-PAGINATE	$O(P\ell + U \log U)$	$O(U)$	14	analytics.rs:1096
DIFF-HUNTS	$O(2T_{\text{match}} + P)$	$O(P)$	15	analytics.rs:1342
ENCODE-INGESTED-AT	$O(1)$	$O(1)$	3	relation.rs:151
CHECK-STEP-COMPAT	$O(\Sigma)$	$O(1)$	8	catalog/mod.rs:105

Cuadro A.1: Resumen de complejidad. n = vértices, m = aristas, $\bar{d}$ = grado promedio, $d(v)$ = grado de v , ℓ = cantidad de pasos de la hipótesis, k = k -simplicidad, n_s = cantidad de vértices de partida candidatos, $\bar{b}$ = factor de branching candidato promedio, W = cantidad de workers rayon, K = cap top- K , L = spill limit, S = sources muestreados, I = cantidad de iteraciones, P = caminos pre-dedup, U = caminos post-dedup, T = triples (src, rel, dst) únicos.

Apéndice B

Referencia del catálogo

Los 29 patrones de hipótesis ATT&CK built-in. Cada entrada vive como un YAML separado en `graph\_hunter\_core/src/catalog/builtin/cat-NNN.yaml` y se valida + cachea en el primer acceso (Capítulo 25).

ID	MITRE	Patrón	k
cat-001	T1078	User → Host → Process	1
cat-002	T1059.001	User → Process → Process → File	1
cat-003	T1021.001	IP → Host → User → Process	1
cat-004	T1003	User → Process → File	1
cat-005	T1071	Process → Domain → IP	1
cat-006	T1569.002	User → Process → Service	1
cat-007	T1053.005	Process → File → Process	1
cat-008	T1055	Process → Process → File	1
cat-009	T1048.003	User → Process → Domain	1
cat-010	T1204.002	Process → File → Process	1
cat-011	T1547.001	Process → Registry → Process	1
cat-012	T1021	User → Host → Process → File	1
cat-013	T1071	Process → Domain → IP → Process (ciclo)	2
cat-014	T1547.001	Process → Registry → Process (ciclo)	2
cat-015	T1021	Host → User → Process → Host (ciclo)	2
cat-016	T1059	Process → Process → Process (ciclo)	2
cat-017	T1110.003	User {status=Failure} → IP HAVING count(distinct IP) ≥ 3 WITHIN 24h	1
cat-018	T1087.004	User {app~PowerShell} → IP	1
cat-019	T1078.004	User {risk_tag=anonymizing} → IP	1
cat-020	T1110.003	User {status=Success} → IP HAVING count(distinct IP) ≥ 2	1
cat-021	T1078.004	User → IP HAVING count(distinct IP) ≥ 2 WITHIN 6h	1
cat-022	T1528	User → OAuthApp	1
cat-023	T1098.003	User → ServicePrincipal	1
cat-024	T1098.003	User → Role → User	1
cat-025	T1564.008	User → MailboxRule → Domain	1
cat-026	T1218	Process → Process {name~certutil}	1
cat-027	T1014	Process → File {path~.sys}	1
cat-028	T1562.001	Process → Service {name~Sense}	1
cat-029	T1053.005	User → Host → Task → Process	1

Cuadro B.1: Catálogo built-in. Las flechas omiten el tipo de relación por compacidad; ver el YAML para los patrones exactos.

Las entradas 017–021 ejercitan la gramática de predicados de metadata; 022–025 son los patrones de identidad cloud que requieren tipos de relación **Consent**, **Create**, **Assign**, **Forward**; 026–028 son los patrones de modernización de endpoint que usan matchers de substring sobre metadata del vértice destino.

Apéndice C

Benchmarks

Dos suites de benchmarks Criterion viven en `graph\_hunter\_core/benches/`:

- `hunt\_latency.rs` — timing end-to-end de `search\_temporal\_pattern` sobre un grafo sintético. Se usa como portón de “ $\leq 5\%$ de regresión” en los planes de refactor P1/P2.
- `dedup\_throughput.rs` — timing de ingesta del `SpillableEdgeStore`. Valida el claim de RAM $O(k)$ del merge k -way sobre una muestra de 2M records (delta de 0.23 MB vs los ~ 92 MB del approach basado en Vec anterior).

C.1. Ejecución

```
cd graph_hunter_core
cargo bench --bench hunt_latency
cargo bench --bench dedup_throughput
```

El output default de Criterion aterriza en `graph\_hunter\_core/target/criterion/` como reportes HTML.

C.2. Baseline actual (ilustrativo)

Benchmark	Resultado	Notas
hunt_latency 2-pasos, 10K aristas	$\sim 200\ \mu\text{s}$	Rust escalar
hunt_latency 2-pasos, 10K aristas, SIMD	$\sim 70\ \mu\text{s}$	libgraphmatch
dedup_throughput 2M records	$\sim 1.5\ \text{s}$	modo spill

Cuadro C.1: Números de benchmark ilustrativos en un CPU laptop-class (AMD Ryzen 7 5800H). Rerunear localmente para números al día; los valores reales varían por hardware y flags de build.

C.3. Comparación three-way SIMD (planeada)

El follow-up P2-D agrega una tercera columna a la tabla de hunt-latency: `simd\_rust` puro-Rust. La comparación debería mostrar (a) escalar, (b) FFI libgraphmatch, (c) intrínsecos Rust. La expectativa es que los intrínsecos Rust alcancen ~ 70 – 80% del peak libgraphmatch, momento en el cual el beneficio operacional de sacar el toolchain C++ domina.

Apéndice D

Índice de fuente

Punteros archivo: línea para cada algoritmo nombrado en el documento.

D.1. `graph_hunter_core/src/`

- `anomaly.rs:148` — ANOMALY-FINALIZE
- `anomaly.rs:191` — COMPUTE-NC
- `anomaly.rs:241` — SCORE-PATH (v1)
- `anomaly.rs:351` — `node\_anomaly\_estimate`
- `analytics.rs:289` — `build\_narrative`
- `analytics.rs:641` — COMPUTE-BETWEENNESS
- `analytics.rs:746` — TEMPORAL-PAGERANK
- `analytics.rs:1096` — SCORE-AND-PAGINATE
- `analytics.rs:1289` — `get\_relation\_schema`
- `analytics.rs:1342` — DIFF-HUNTS
- `backend/mod.rs` — `GraphRead`, `GraphWrite`
- `backend/in\_memory.rs` — impls del trait
- `catalog/mod.rs:105` — CHECK-STEP-COMPAT
- `catalog/loader.rs` — carga YAML + validación
- `dsl.rs:47` — PARSE-DSL
- `graph.rs:67` — struct `InMemoryGraph`
- `graph.rs:211` — ADD-ENTITY
- `graph.rs:227` — ADD-RELATION
- `graph.rs:437` — SORT-EDGES-BY-TIMESTAMP
- `graph.rs:461` — SEARCH-TEMPORAL-PATTERN
- `graph.rs:689` — SEARCH-TEMPORAL-PATTERN-SIMD
- `graph.rs:782` — SEARCH-TEMPORAL-PATTERN-SMART
- `graph.rs:932` — DFS-MATCH-SMART
- `graph.rs:1121` — DFS-MATCH-ITERATIVE
- `hypothesis.rs` — `Hypothesis`, `HypothesisStep`, `Predicate`
- `ingest/batch.rs` — `ParsedBatch`

- `ingest/channels.rs` — canales acotados
- `ingest/enrichment.rs` — preempción de enriquecimiento
- `ingest/overflow.rs` — persistencia de cola
- `ingest/parser\_task.rs` — task parser
- `ingest/source\_poller.rs` — loop de polling
- `ingest/writer.rs` — `GraphWriter`
- `ingest\_adapter.rs` — `IngestAdapter`, `LogSourceAdapter`
- `interner.rs:60` — `INTERN`
- `interner.rs:74` — `RESOLVE`
- `metadata\_store.rs` — append / read de metadatos
- `relation.rs:83` — struct `CompactRelation`
- `relation.rs:151` — `ENCODE-INGESTED-AT`
- `scoring/mod.rs` — trait `ScoreComponent`
- `scoring/composite.rs` — `CompositeScorer`
- `scoring/components.rs` — 5 componentes
- `simd\_matcher.rs` — FFI `libgraphmatch`
- `simd\_rust.rs` — `INTERSECT-SORTED-U32`
- `spill.rs:327` — `K-WAY-MERGE` finalize
- `spill.rs:524` — `GET-EDGES`
- `sources/mod.rs:275` — trait `LogSource`

D.2. `graph_hunter_api/src/`

- `lib.rs` — fachada `GraphHunterApi`, `ApiState`
- `dto/*.rs` — DTOs request/response
- `operations/*.rs` — operaciones por categoría
- `state/session.rs` — `Session`, `SessionStore`, `SessionHandle`
- `state/persistence.rs` — load/save de sesión
- `state/session\_types.rs` — `SessionFile`, `Note`, `EntityTag`
- `tests/parity.rs` — entry del arnés
- `tests/parity/dto\_shapes.rs` — Fase A
- `tests/parity/scenarios.rs` — Fase B
- `tests/parity/fixture.rs` — constructores de fixture

D.3. `app/src-tauri/`

- `src/commands/` — módulos de comandos Tauri
- `src/http\_api.rs` — handlers HTTP
- `src/lib.rs` — registro en el Builder Tauri
- `src/tauri\_emitter.rs` — `TauriEventEmitter`

D.4. graph-hunter-mcp/

- `src/index.ts` — todos los handlers de tool MCP

D.5. libgraphmatch/

- `include/graphmatch/csr\_graph.hpp` — `CSRTemporalGraph` (`add\_node`, `add\_edge`, `finalize`)
- `src/simd\_intersect.cpp` — intersección AVX2 de conjuntos ordenados
- `src/matcher.cpp` — entry del matcher SIMD
- `tests/test\_csr\_graph.cpp` — tests de límites en C++ (incluyendo la suite `add\_edge` de `boundaries P2-D`)